SIEM Detection Engineering & Threat Detection

Practical Implementation Report Using Elastic Stack 9.x

1. Introduction

In the modern digital landscape, organizations face an increasingly sophisticated and persistent threat landscape. Cybersecurity and continuous security monitoring have become foundational pillars for protecting enterprise infrastructure, intellectual property, and sensitive data against malicious actors. As cyber attacks grow in velocity and complexity, perimeter defenses alone are no longer sufficient, making proactive visibility and rapid threat detection essential components of organizational defense posture.

Security Information and Event Management (SIEM) systems play a central role in modern Security Operations Centers (SOCs). By centralizing log collection across endpoints, servers, network devices, and cloud services, a SIEM normalizes and aggregates vast volumes of telemetry data. This centralized repository enables security analysts to correlate disparate events, uncover hidden attack patterns, and execute timely incident investigations.

Complementing log collection is the critical discipline of detection engineering. Detection engineering focuses on designing, testing, and refining security rules and analytics to identify suspicious activities and anomalous behaviors with high fidelity. Effective detection engineering bridges the gap between raw telemetry and actionable security alerts, minimizing false positives while ensuring that genuine threats are surfaced immediately.

This project leverages Elastic Stack 9.x, an industry-leading analytics and security platform, to build a fully functional SIEM environment. Elastic Stack provides robust capabilities for log ingestion through Elastic Agent and Filebeat, rapid search and indexing via Elasticsearch, and advanced visualization and rule management through Kibana. Utilizing Elastic Search Query Language (KQL), custom detection rules are crafted to monitor system events and identify adversary tactics, techniques, and procedures (TTPs).

To evaluate the effectiveness of the implemented detection engineering framework, this project operates within a controlled lab environment. By simulating real-world attack scenarios—such as privilege escalation, brute-force attempts, and unauthorized scheduled task creation—from an attacker system (Kali Linux) against a target enterprise environment (Windows Server 2019), the project demonstrates end-to-end detection engineering workflows from log ingestion to automated alert investigation.

2. Project Objectives

The primary objectives of this practical implementation project are structured to encompass the full lifecycle of a SIEM deployment and threat detection engineering workflow:

1. Implement a centralized SIEM environment using Elastic Stack 9.x.
2. Collect and analyze security logs from Windows Server 2019 and Kali Linux.
3. Develop custom detection rules using KQL (Kibana Query Language).
4. Simulate different security events and attack techniques in a controlled environment.
5. Generate and investigate automated security alerts.
6. Evaluate detection effectiveness and identify potential false positives and false negatives.
7. Gain practical experience in SOC monitoring and incident investigation.

3. Project Scope – Use Cases

To establish a comprehensive detection engineering framework, the project scope encompasses six distinct practical detection use cases. These use cases target prevalent attack vectors and Windows security event categories commonly observed during enterprise security breaches. The six detection use cases are implemented in the exact sequence specified below:

1. Suspicious Scheduled Task Creation Detection: Monitoring Event ID 4698 and related task scheduling mechanisms to detect persistent backdoor installations and unauthorized background execution.

2. Brute Force Attack Detection: Analyzing Windows authentication failures (Event ID 4625) and SSH authentication logs to identify credential stuffing and password guessing campaigns.

3. Suspicious PowerShell Activity Detection: Inspecting PowerShell script block logging (Event ID 4104) and command-line execution for obfuscated code, encoded commands, and malicious download cradles.

4. Privilege Escalation Detection: Tracking security group modifications, token manipulation, and unauthorized user privilege assignments indicating lateral movement and domain escalation.

5. File Integrity Monitoring (FIM): Configuring FIM policies to detect unauthorized creation, modification, or deletion of critical system files, binaries, and configuration stores.

6. Windows Account Creation Detection: Detecting unauthorized local or domain user account creation (Event ID 4720) to identify persistence mechanisms established by threat actors.

4. Lab Environment

The project is executed within an isolated, virtualized laboratory architecture designed to replicate an enterprise network while providing complete safety and control during attack simulations. The underlying infrastructure components and their respective roles within the architecture are detailed in the table below.

Component	Description
SIEM Platform	Elastic Stack 9.x
SIEM Server	Ubuntu (Elasticsearch, Kibana, Fleet Server)
Target System	Windows Server 2019 (Active Directory & Endpoint)
Attacker / Testing System	Kali Linux (Advisory Simulation & Attack Framework)
Log Collection	Filebeat / Elastic Agent
Log Analysis	Kibana Security & Dashboards
Detection Language	KQL (Kibana Query Language)

5. Practical Implementation and Detection Use Cases

The following sections document the complete implementation, configuration, attack simulation, log analysis, and alert investigation results for the six detection use cases outlined in the project scope.

SIEM Detection Engineering: Suspicious Scheduled Task Creation Detection

Platform: *Elastic Stack 9.x* | ***Target OS:*** *Windows Server 2019* | ***Log Collection:*** *Filebeat 9.5.5* | ***SIEM Server:*** *Ubuntu* | ***Detection Method:*** *Custom KQL-Based Detection* | ***Event ID:*** *4698* | ***MITRE ATT&CK:*** *T1053.005*

1. Scenario

This use case demonstrates the detection of suspicious scheduled task creation on Windows Server 2019 using Elastic SIEM. A controlled test was performed by creating scheduled tasks through PowerShell to simulate a potential persistence technique. Windows Security Event ID 4698 was generated and collected by Filebeat, then forwarded to Elastic Stack for analysis. A custom KQL detection rule was configured to identify newly created scheduled tasks and automatically generate security alerts. The generated alert was investigated to validate the detection process and confirm the activity as authorized testing.

2. Lab Environment

The technical environment configured for this detection engineering lab consists of the following enterprise security and operating system components:

- Windows Server 2019
- Elastic SIEM 9.x
- Filebeat 9.5.5
- Sysmon
- Windows Security Event Logs

3. Sysmon Installation and Verification

System Monitor (Sysmon) is deployed on the Windows Server 2019 endpoint to capture advanced system activity, process creations, and driver-level telemetry.

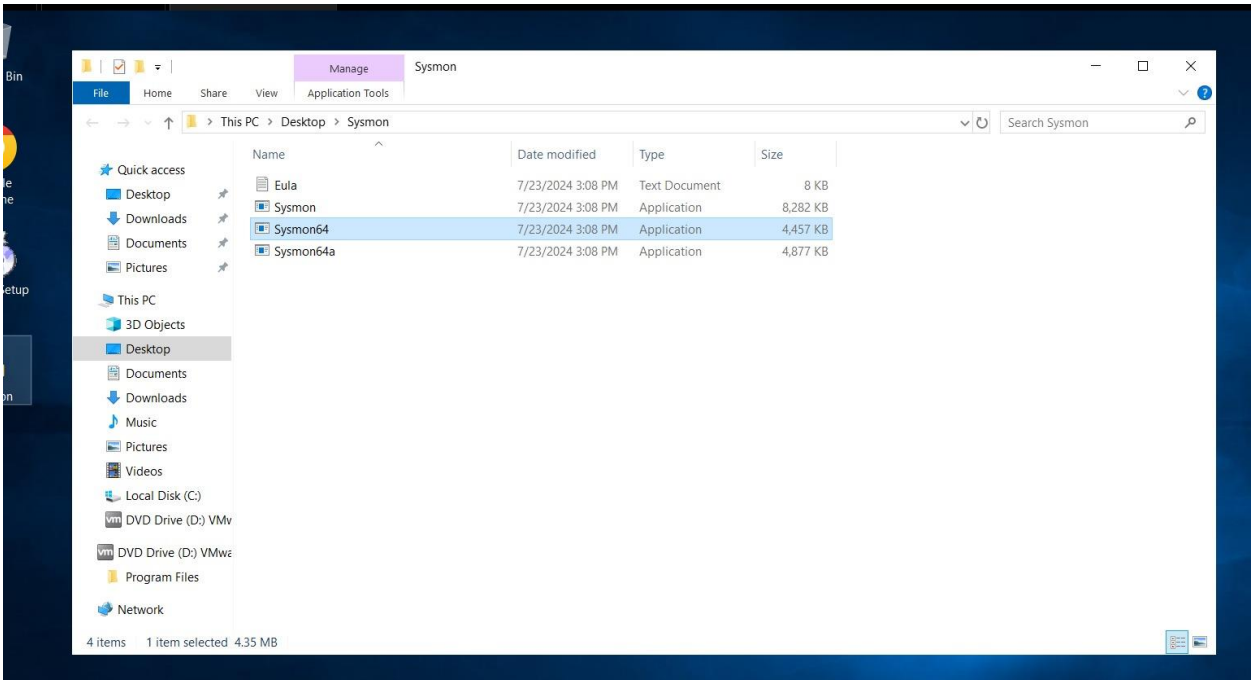


Figure: Preparing Sysmon installation files on Windows Server 2019.

The Sysmon installation package was extracted to the local desktop directory containing binary executables and the End User License Agreement.

```
Sysmon64.exe -accepteula -i
```

This command installs Sysmon and accepts the license agreement automatically.

```
sc query Sysmon64
```

This command verifies that the Sysmon64 service is running.

```
C:\Users\Administrator>cd /d C:\Users\Administrator\Desktop\Sysmon

C:\Users\Administrator\Desktop\Sysmon>Sysmon64.exe -accepteula -i


System Monitor v15.15 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2024 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.

C:\Users\Administrator\Desktop\Sysmon>sc query Sysmon64

SERVICE_NAME: Sysmon64
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                               (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0

C:\Users\Administrator\Desktop\Sysmon>
```

Figure: Executing Sysmon installation via Command Prompt.

4. Windows Event Logging Configuration

To ensure proper visibility into system-level events and security auditing, Windows Event Channels and advanced audit policies were configured.

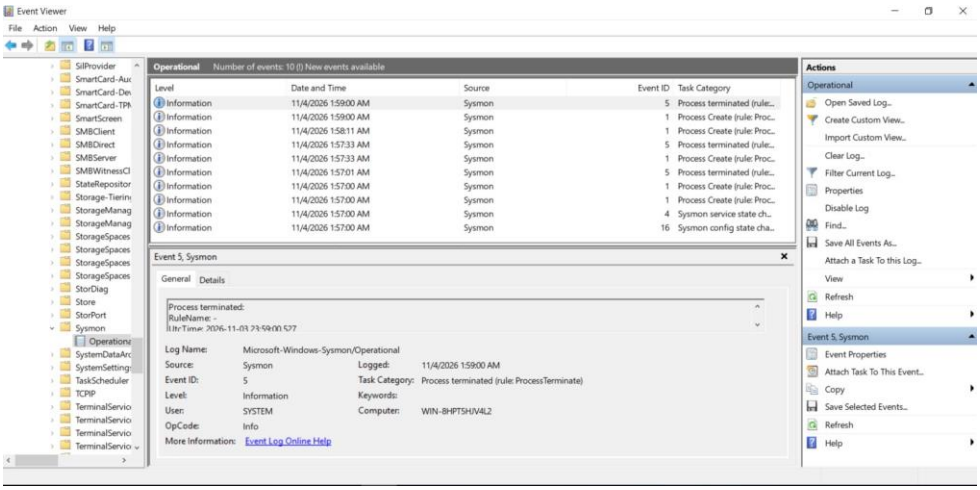


Figure: Verifying that the Sysmon Operational event log is available and receiving events.

```
auditpol /set /subcategory:"Other Object Access Events" /success:enable
/failure:enable
```

This command enables successful and failed auditing for Other Object Access Events, allowing scheduled task creation events to be recorded.

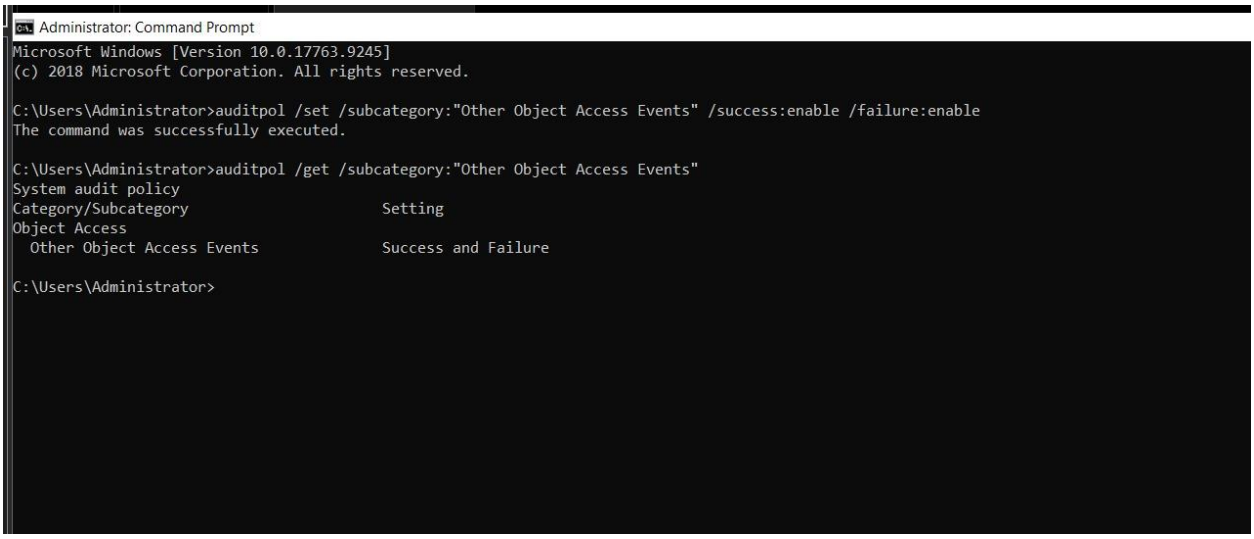


Figure: Configuring and verifying Windows audit policy settings.

5. Scheduled Task Creation and Log Collection

Persistence via scheduled tasks is a common technique utilized by threat actors. Test scheduled tasks were created to simulate this behavior and generate audit logs.

```
$Action = New-ScheduledTaskAction -Execute "notepad.exe"
$Trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(30)
Register-ScheduledTask -TaskName "SOC_Test_Task" -Action $Action -Trigger $Trigger -
Description "SIEM Detection Testing"
```

A test scheduled task was created to generate a Windows Security Event ID 4698.

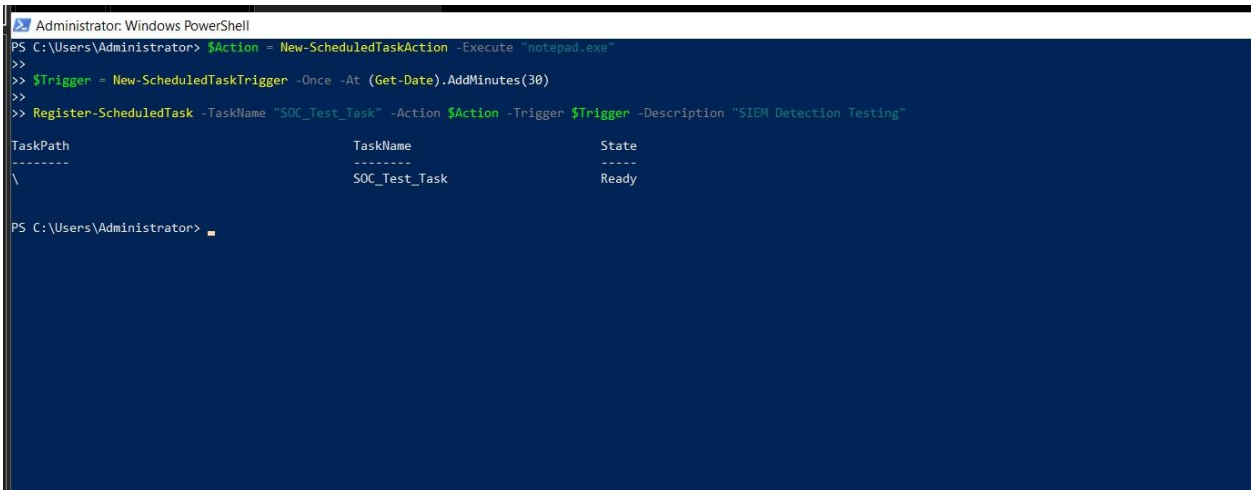


Figure: Executing PowerShell script to register the first test scheduled task.

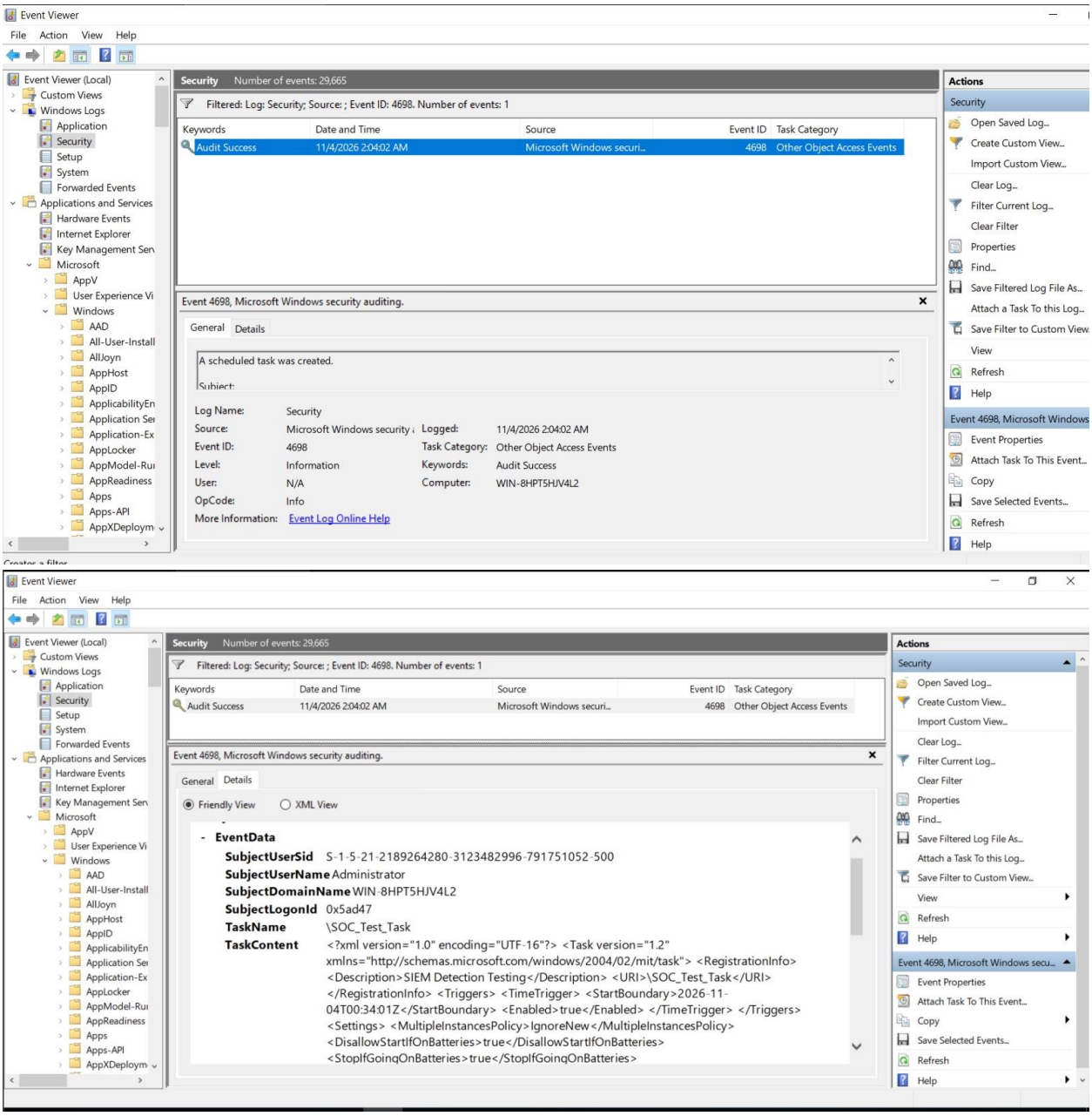


Figure: Confirming that Windows generated Event ID 4698 after scheduled task creation.

```
event.code: "4698"
```

This query searches for scheduled task creation events collected by Elastic.

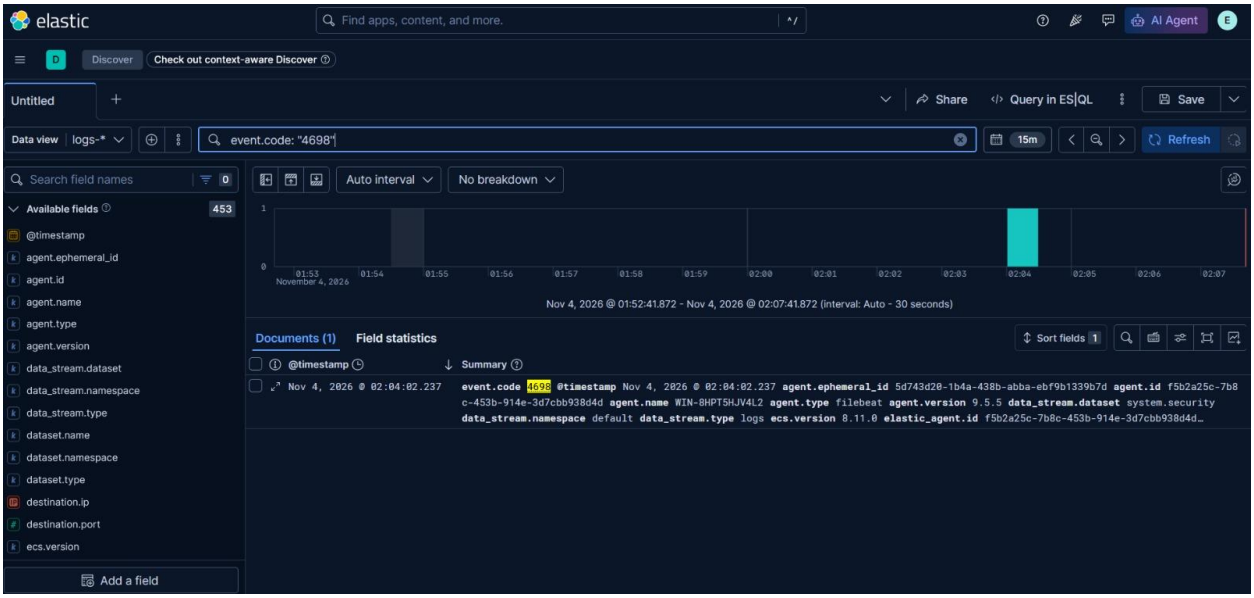


Figure: Elastic Discover interface showing the ingested Windows scheduled task creation event.

6. Detection Rule Configuration

A custom detection rule was authored within Elastic Security to automatically identify scheduled task creation events across monitored endpoints.

Rule Configuration Parameters

Rule name: Suspicious Scheduled Task Creation

Rule type: Custom Query

Severity: Medium

Risk score: 47

Tags: Persistence, Windows, MITRE T1053.005

```
event.code: "4698"
```

The custom detection rule monitors Windows scheduled task creation events.

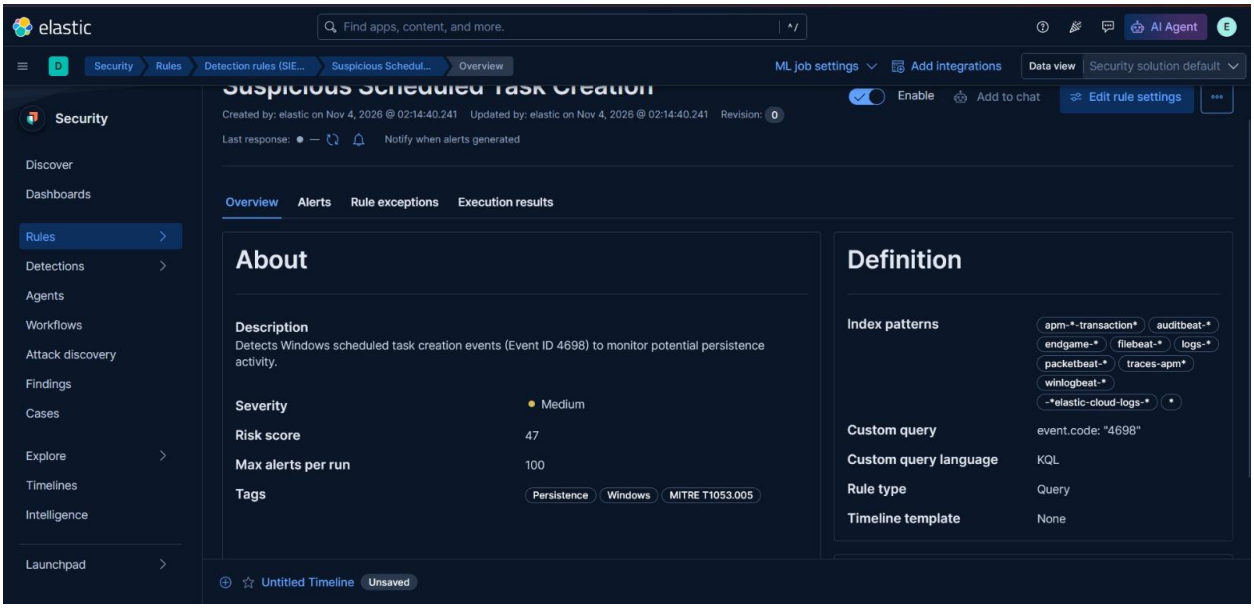


Figure: Configuring custom detection rule parameters and KQL query in Elastic Security.

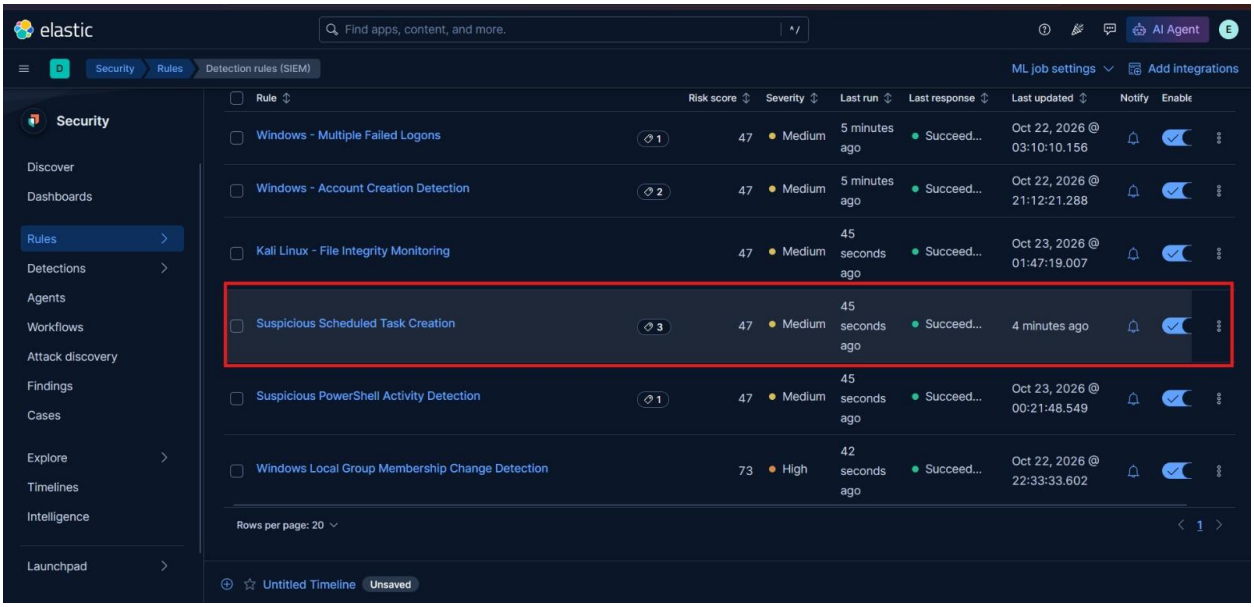


Figure: Verifying that the detection rule is saved and enabled with its configured schedule.

7. Detection Testing and Alert Generation

Subsequent validation activities were executed to verify that the configured detection rule correctly triggers upon encountering new scheduled task creation events.

```
$Action = New-ScheduledTaskAction -Execute "notepad.exe"
$Trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(45)
Register-ScheduledTask -TaskName "SOC_Test_Task_02" -Action $Action -Trigger
$Trigger -Description "SIEM Detection Rule Validation"
```

A second scheduled task was created to validate the detection rule.

```
PS C:\Users\Administrator> $Action = New-ScheduledTaskAction -Execute "notepad.exe"
>> $Trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(45)
>> Register-ScheduledTask -TaskName "SOC_Test_Task_02" -Action $Action -Trigger $Trigger -Description "SIEM Detection Rule Validation"

TaskPath                                 TaskName                    State
--------                                 --------                    -----
\                                        SOC_Test_Task_02            Ready


PS C:\Users\Administrator>
```

Figure: Creating the second test scheduled task via PowerShell.

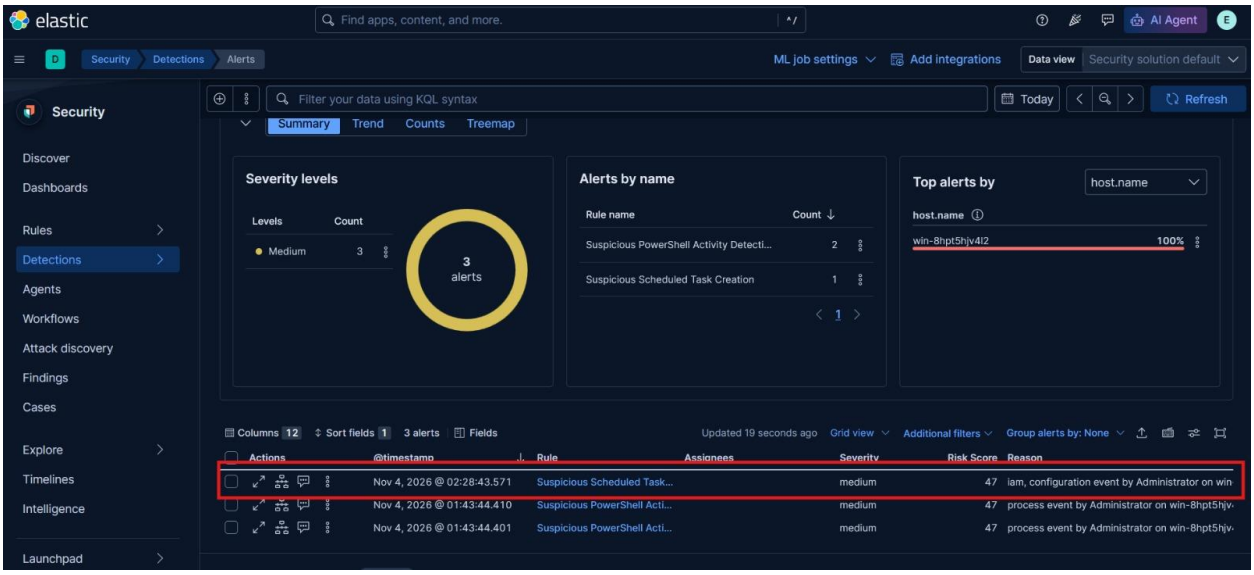


Figure: Elastic successfully generated an alert after detecting the new scheduled task creation event.

8. Alert Investigation and Triage

Security analysts performed a comprehensive triage of the generated alert within the Elastic Security SIEM platform to inspect associated metadata and event fields.

Alert Associated Fields

Event ID: 4698

Host: WIN-8HPT5HJV4L2

User: Administrator

Task name: SOC_Test_Task_02

Executable: notepad.exe

The alert was investigated by reviewing the original event and its associated fields.

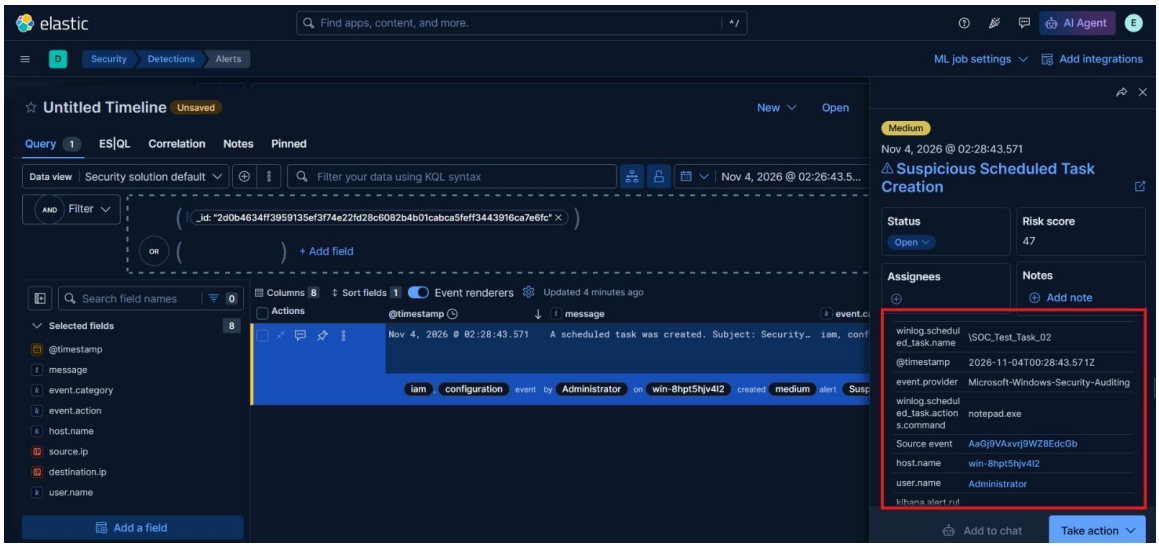


Figure: Detailed alert investigation view displaying event parameters and metadata.

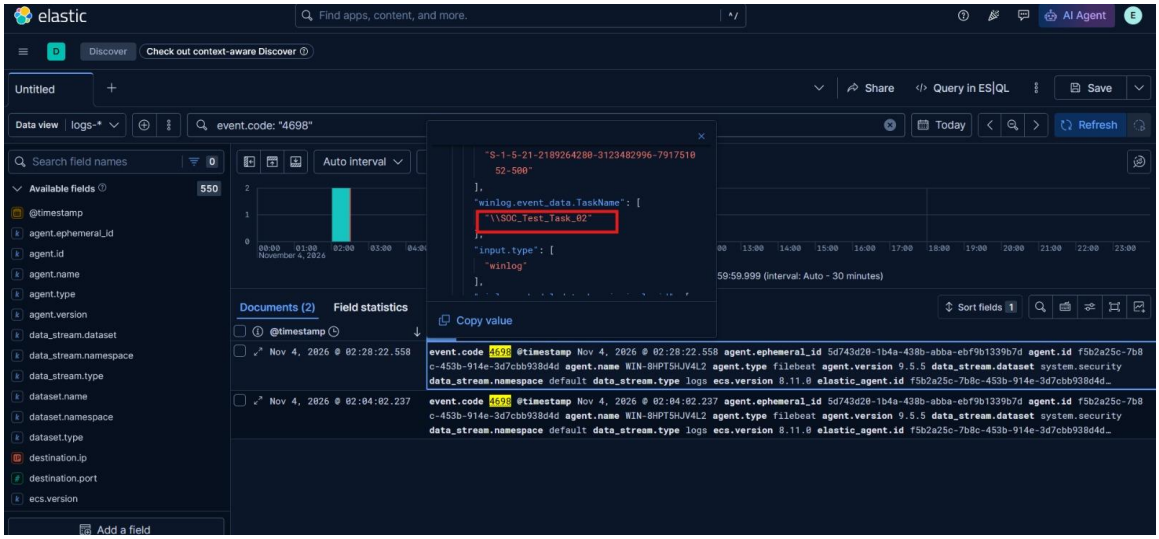


Figure: Confirming the original Windows event associated with the generated alert in Elastic Discover.

9. Investigation Result

Summary of findings and investigation outcomes resulting from the detection engineering and triage workflow:

Detection Status: Successful

Alert Generation: Successful

Log Collection: Successful

Triage Result: Benign Activity – Authorized Testing

MITRE ATT&CK: T1053.005 – Scheduled Task/Job: Scheduled Task

Clarify that the test activity was authorized and performed for validation purposes. The rule detects task creation events but does not independently determine whether they are malicious.

10. Conclusion

The use case successfully demonstrated Windows scheduled task event collection, SIEM detection, alert generation, and investigation using Elastic SIEM.

SIEM Detection Engineering: Brute Force Attack Detection Using Elastic Stack

Platform: Elastic Stack 9.x | Target OS: Windows Server 2019 | Attacker: Kali Linux | SIEM Server: Ubuntu | Detection Method: Threshold-Based Detection

1. Scenario Description

The project demonstrates a controlled SMB brute-force simulation against a Windows Server 2019 machine using Kali Linux. Three virtual machines (Kali Linux, Windows Server, and Ubuntu) operate within the same lab network, while Ubuntu hosts Elastic Stack 9.x as the centralized SIEM platform. A dedicated test account named siemtest is created on Windows Server. Five incorrect SMB authentication attempts are performed from Kali Linux, generating Windows Security Event ID 4625. Filebeat forwards the events to Elastic Stack, where a threshold-based detection rule generates an alert after five failed attempts within five minutes.

2. Lab Network Configuration and Connectivity

The lab consists of three virtual machines connected through a shared virtual network. The IP addresses of each machine were verified to confirm network configuration and establish the communication environment required for the detection scenario.

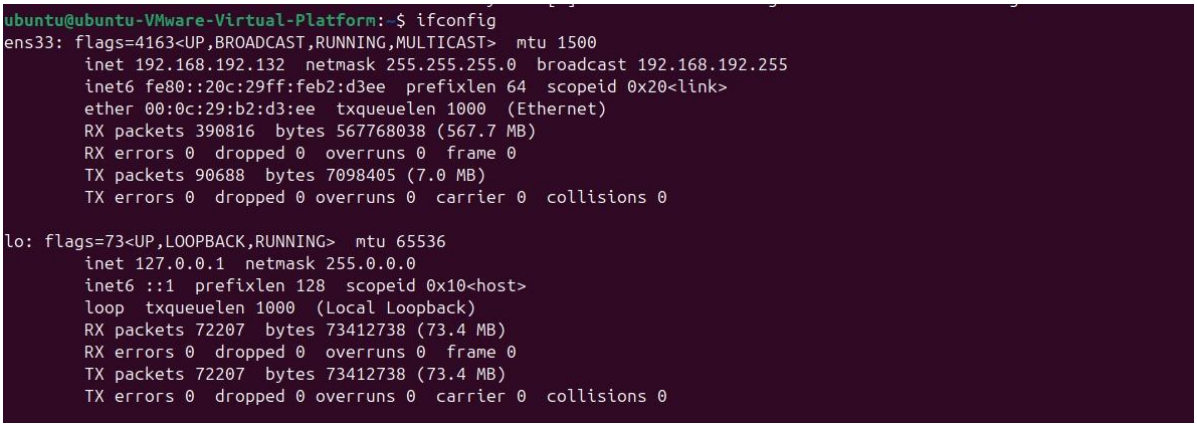


Figure 1 – Ubuntu SIEM Server Network Configuration

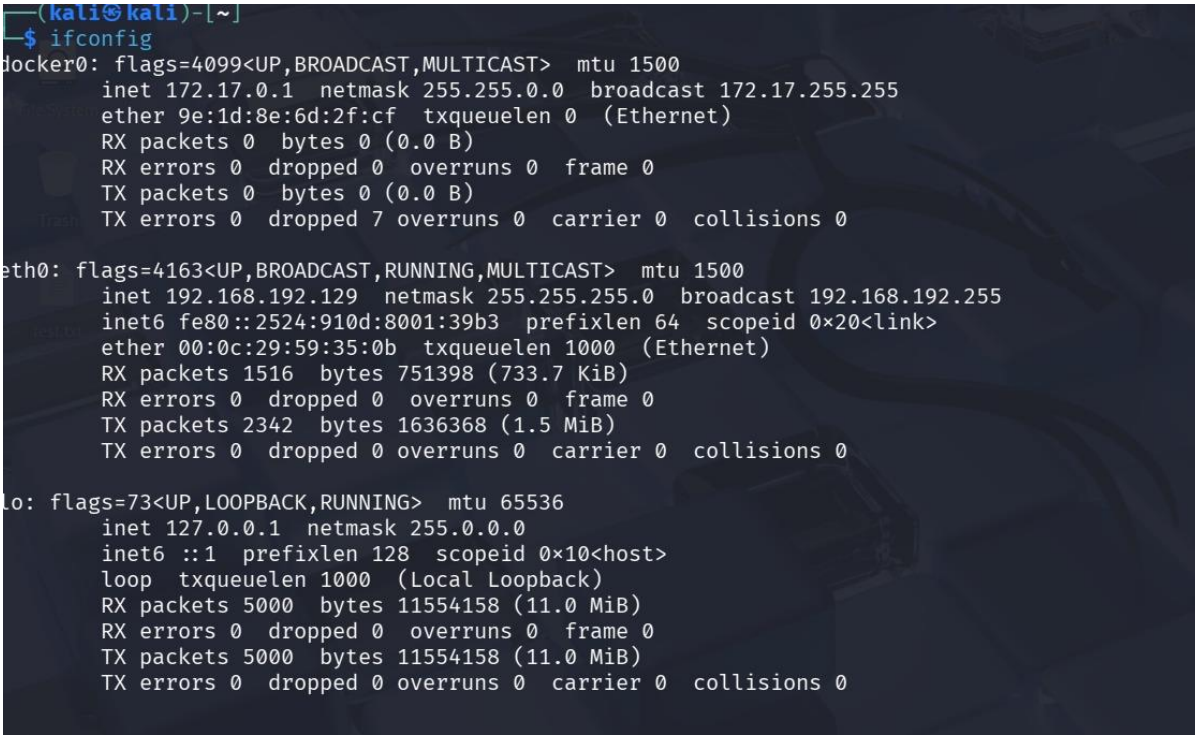


Figure 2 – Kali Linux Attacker Network Configuration

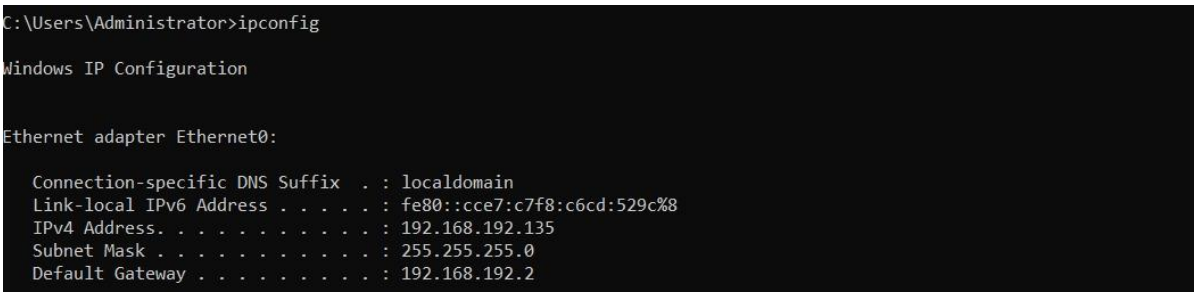


Figure 3 – Windows Server Target Network Configuration

3. Test Account Creation

A dedicated test account named siemtest was created on Windows Server to provide a controlled account for SMB authentication testing.

```
net user siemtest /add
```



Figure 4 – Creation of the Dedicated Test Account

4. Log Discovery and Verification

After the failed authentication attempts, Windows Security Event ID 4625 was collected and forwarded to Elastic Stack through Filebeat. Kibana Discover was used to search and verify the incoming failed authentication events.

```
event.code: "4625" AND data_stream.dataset: "system.security"
```

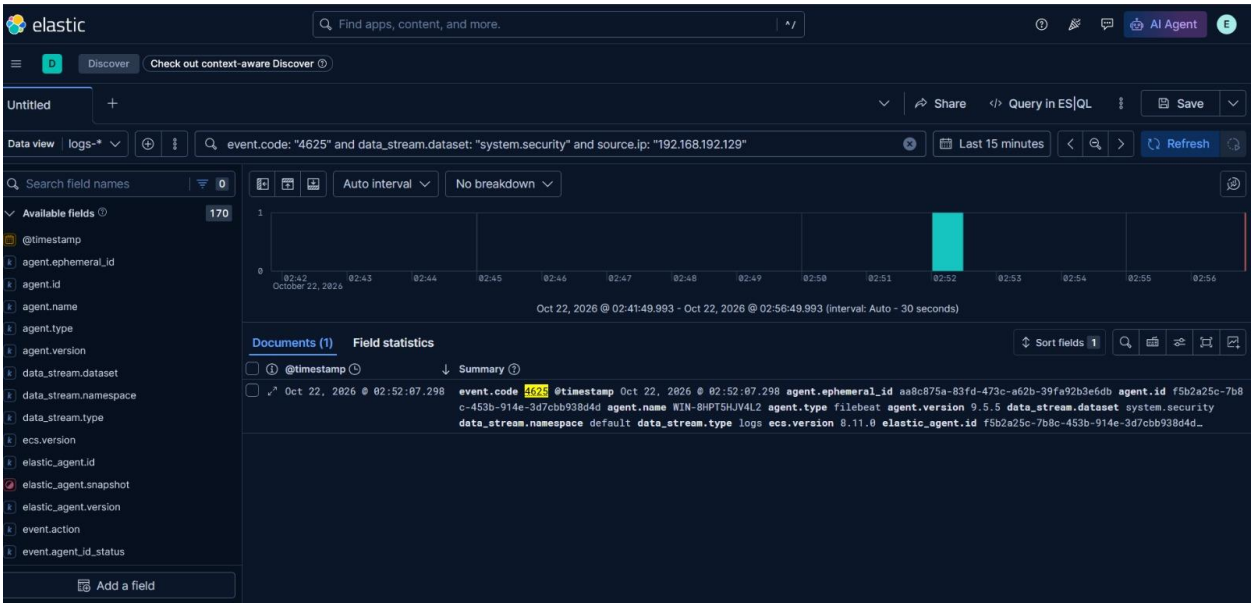


Figure 5 – Searching for Failed Authentication Events in Kibana Discover

5. Detection Rule Configuration

A custom threshold-based detection rule was configured in Elastic Security to identify repeated failed authentication attempts. The rule was designed to generate an alert when five or more failed login events occurred within five minutes, grouped by source IP address.

- **Rule Name:** Windows - Multiple Failed Logons
- **Rule Type:** Threshold
- **Group By:** source.ip
- **Threshold:** 5 events
- **Time Window:** 5 minutes
- **Severity:** Medium
- **Risk Score:** 47

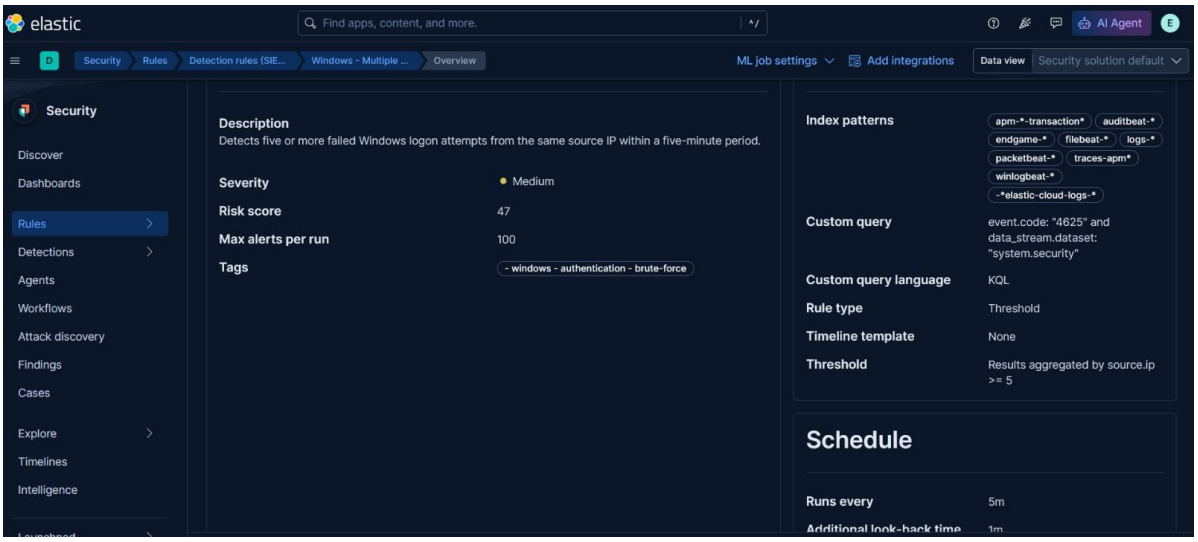


Figure 6 – Brute Force Detection Rule Threshold Configuration

6. Detection Rule Execution

The configured detection rule was enabled in Elastic Security to evaluate incoming authentication events continuously and identify activity matching the specified threshold conditions.

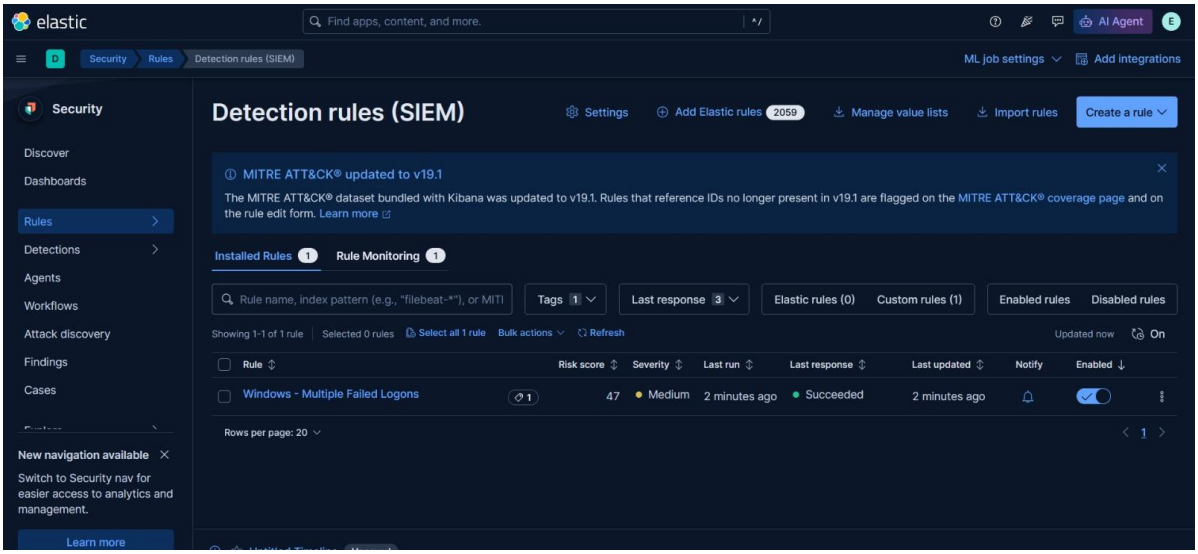


Figure 7 – Enabled Brute Force Detection Rule

7. Attack Simulation from Kali Linux

The SMB authentication testing was performed from Kali Linux against the Windows Server target using the dedicated siemtest account. An incorrect password was supplied repeatedly to generate failed authentication events.

```
smbclient -L //192.168.192.135 -U siemtest
```

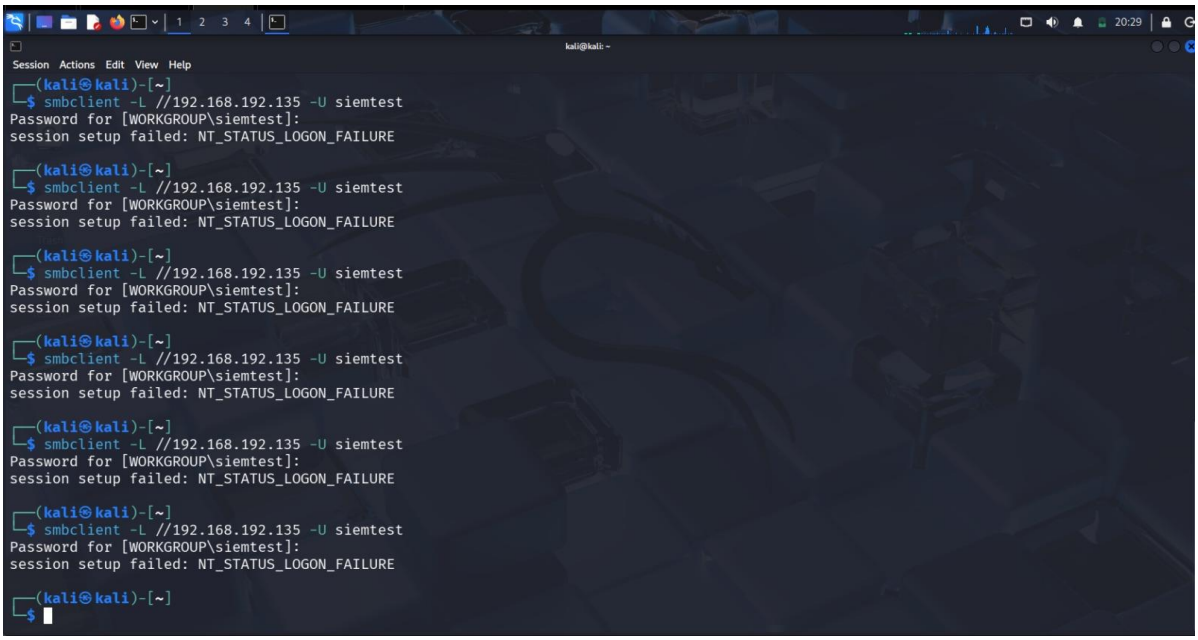


Figure 8 – Simulated Failed SMB Authentication Attempts

8. Detection Results and Security Alert

After the fifth failed authentication attempt within the configured five-minute window, Elastic Security generated a detection alert. The alert confirmed that the threshold-based detection logic successfully identified the repeated authentication failures.

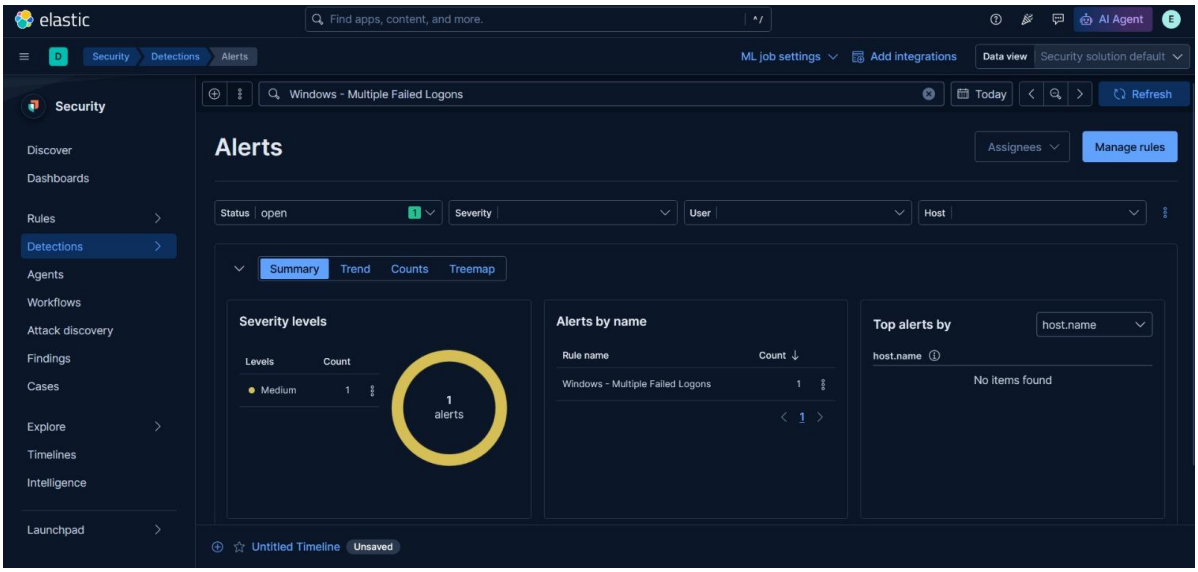


Figure 9 – Successful Brute Force Detection Alert

9. Conclusion

The experiment successfully demonstrated the complete detection workflow: SMB attack simulation, Windows event generation, centralized log collection, threshold-based rule configuration, and security alert generation in Elastic Security. The generated alert indicates suspicious authentication activity and requires thorough security analyst investigation before confirming a security incident.

SIEM Detection Engineering: Suspicious PowerShell Activity Detection Using Elastic Stack

Platform: Elastic Stack 9.x | Target OS: Windows Server 2019 | Log Collection: Filebeat | Log Analysis: Kibana | Detection Method: Custom KQL Detection Rule | Event ID: 4104

1. Report Scenario

This use case demonstrates the detection of suspicious PowerShell activity using Elastic Stack SIEM. PowerShell Script Block Logging (Event ID 4104) is used to monitor executed PowerShell script content. A controlled, non-malicious test was performed on Windows Server 2019 using a Base64 decoding operation to generate script block logs containing the keyword FromBase64String. Filebeat collected and forwarded the logs to Elastic Stack, where a custom KQL detection rule searched for suspicious PowerShell indicators and generated security alerts.

2. Lab Environment

The lab environment components and configuration parameters are summarized below:

Component	Configuration
SIEM Platform	Elastic Stack 9.x
Log Analysis	Kibana
Log Collection	Filebeat
Target Endpoint	Windows Server 2019
Log Source	Microsoft-Windows-PowerShell/Operational
Event ID	4104
Detection Method	Custom KQL Rule

3. PowerShell Test Execution

A controlled PowerShell command was executed to generate a script block containing a Base64 decoding operation. The test was non-malicious and intended to validate script block logging.

```
$start = Get-Date
powershell.exe -NoProfile -Command
"[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String('...'))
| Write-Output"
Start-Sleep -Seconds 3
```

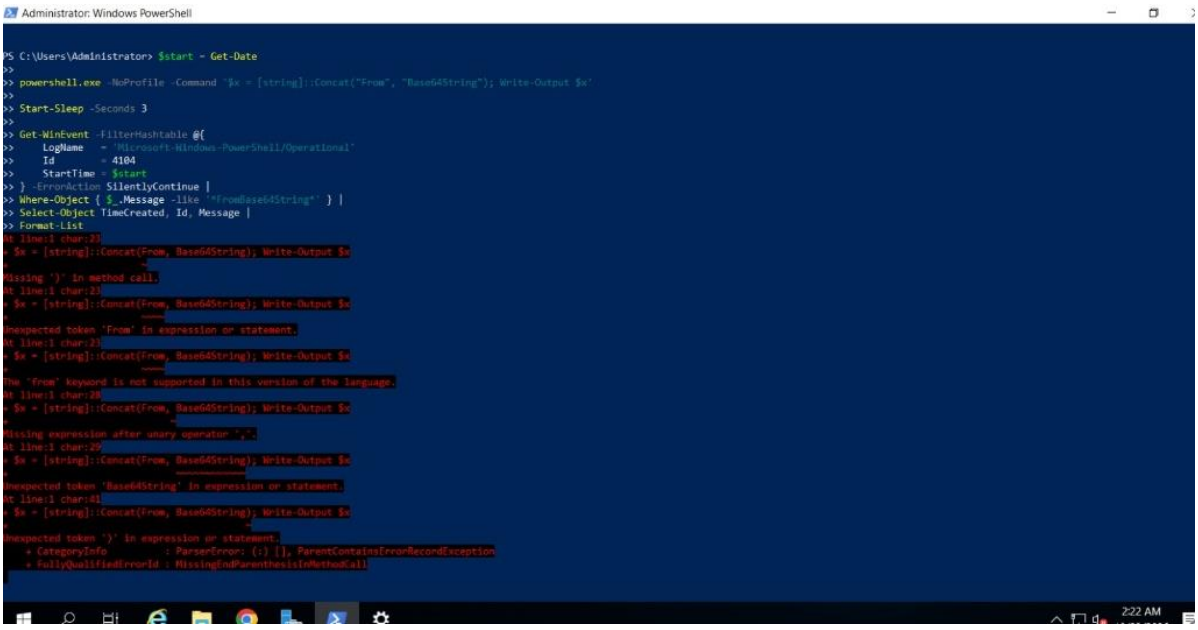


Figure 1 – PowerShell Test Execution

4. Windows Event Log Verification

Windows PowerShell Script Block Logging was inspected to confirm that Event ID 4104 recorded the test execution and the FromBase64String keyword.

```
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -FilterHashtable
@{ Id = 4104; StartTime = (Get-Date).AddMinutes(-5) } -ErrorAction
SilentlyContinue | Where-Object { $_.Message -like "*FromBase64String*" } |
Select-Object TimeCreated, Id, Message | Format-List
```

```
PS C:\Users\Administrator> $start = Get-Date
>>
>> powershell.exe -NoProfile -Command '& { $marker = "FromBase64String"; Write-Output $marker }'
>>
>> Start-Sleep -Seconds 3
>>
>> Get-WinEvent -FilterHashtable @{
>>     LogName   = 'Microsoft-Windows-PowerShell/Operational'
>>     Id        = 4104
>>     StartTime = $start
>> } -ErrorAction SilentlyContinue |
>> Select-Object TimeCreated, Id, Message |
>> Format-List
FromBase64String : The term 'FromBase64String' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that
the path is correct and try again.
At line:1 char:15
+ & { $marker = FromBase64String; Write-Output $marker }
+               ~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (FromBase64String:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException


TimeCreated : 10/23/2026 2:14:59 AM
Id          : 4104
Message     : Creating Scriptblock text (1 of 1):
              $global:?

              ScriptBlock ID: 953e16ac-fd08-4d63-b726-0fb02fe31f82
              Path:

TimeCreated : 10/23/2026 2:14:59 AM
Id          : 4104
Message     : Creating Scriptblock text (1 of 1):
              { Set-StrictMode -Version 1; $_.OriginInfo }

              ScriptBlock ID: 6a07bd8b-af24-48e4-a2fb-1d3edb8ee659
              Path:

TimeCreated : 10/23/2026 2:14:59 AM
Id          : 4104
Message     : Creating Scriptblock text (1 of 1):
              { Set-StrictMode -Version 1; $_.ErrorCategory_Message }
```

Figure 2 – Verification of PowerShell Event ID 4104

5. Kibana Discover – Log Collection and Search

Kibana Discover was used to verify the successful collection and indexing of PowerShell Script Block Logging events.

```
event.code: "4104"
```

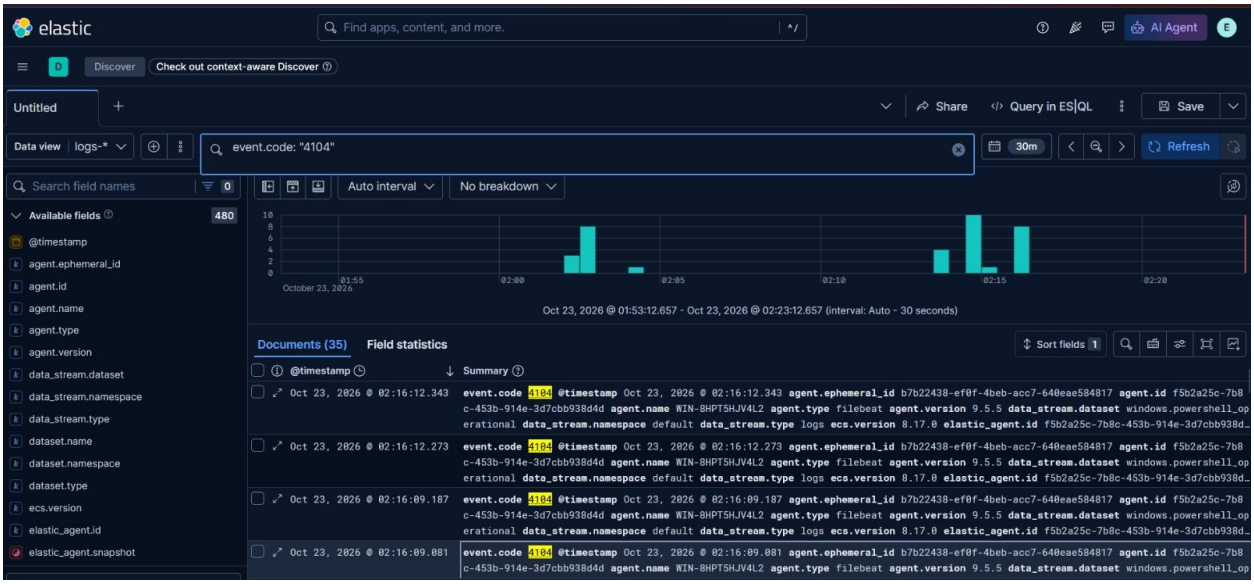


Figure 3 – PowerShell Script Block Events in Kibana Discover

For keyword-specific log analysis, the filtered search query was executed to locate specific script block content matching the decoding indicator.

```
event.code: "4104" and powershell.file.script_block_text: *FromBase64String*
```

- **Total PowerShell logs indexed:** 35
- **Documents matching FromBase64String:** 5

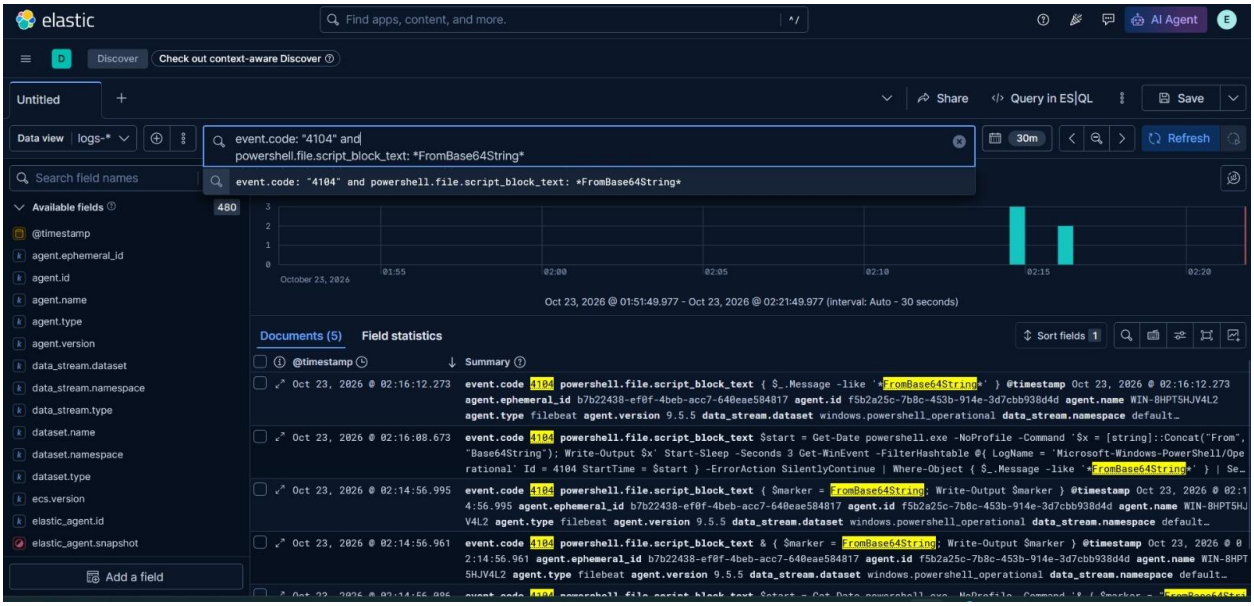


Figure 4 – Filtering PowerShell Events by Suspicious Keyword

6. Custom Detection Rule Configuration

A custom KQL detection rule was created in Kibana Security to identify PowerShell script blocks containing suspicious execution indicators.

```
event.code: "4104" and
powershell.file.script_block_text: (
  *EncodedCommand* or
  *FromBase64String* or
  *DownloadString* or
  *Invoke-Expression* or
  *WindowStyle* or
  *ExecutionPolicy* or
  *Reflection.Assembly*
)
```

- **Rule Name:** Suspicious PowerShell Activity Detection
- **Rule Type:** Custom Query
- **Index Pattern:** logs-*
- **Severity:** Medium
- **Risk Score:** 47
- **Schedule:** Every 1 minute
- **Look-back Time:** 1 minute
- **Event ID:** 4104

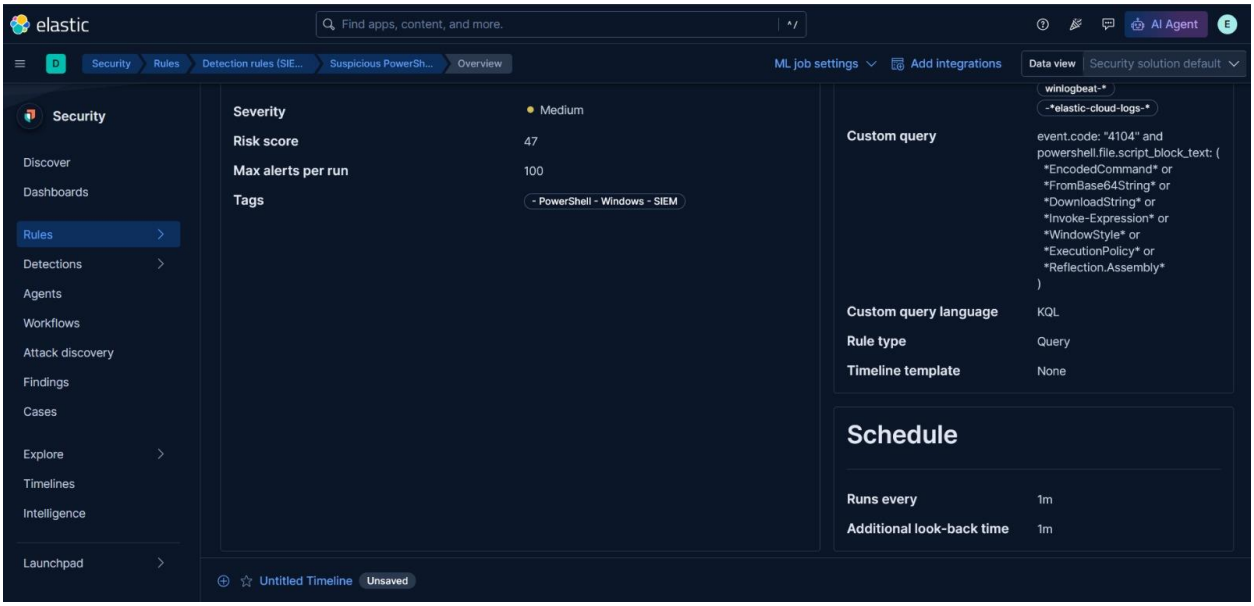


Figure 5 – Suspicious PowerShell Detection Rule Configuration

7. Detection Rule Activation

The configured detection rule was enabled to continuously evaluate incoming PowerShell script block events against the defined suspicious indicators.

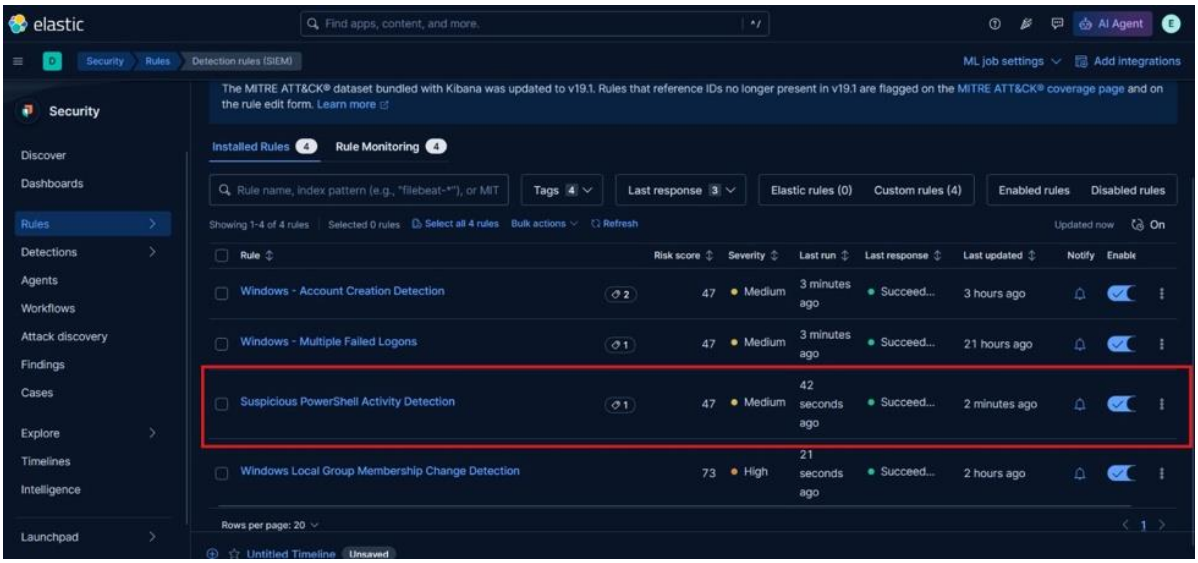


Figure 6 – Enabled Suspicious PowerShell Detection Rule

8. Detection Results and Alerts

The rule successfully identified the test events containing FromBase64String and generated corresponding security alerts.

- **Matching indexed documents:** 5
- **Generated detection alerts:** 5
- **Severity:** Medium
- **Risk Score:** 47

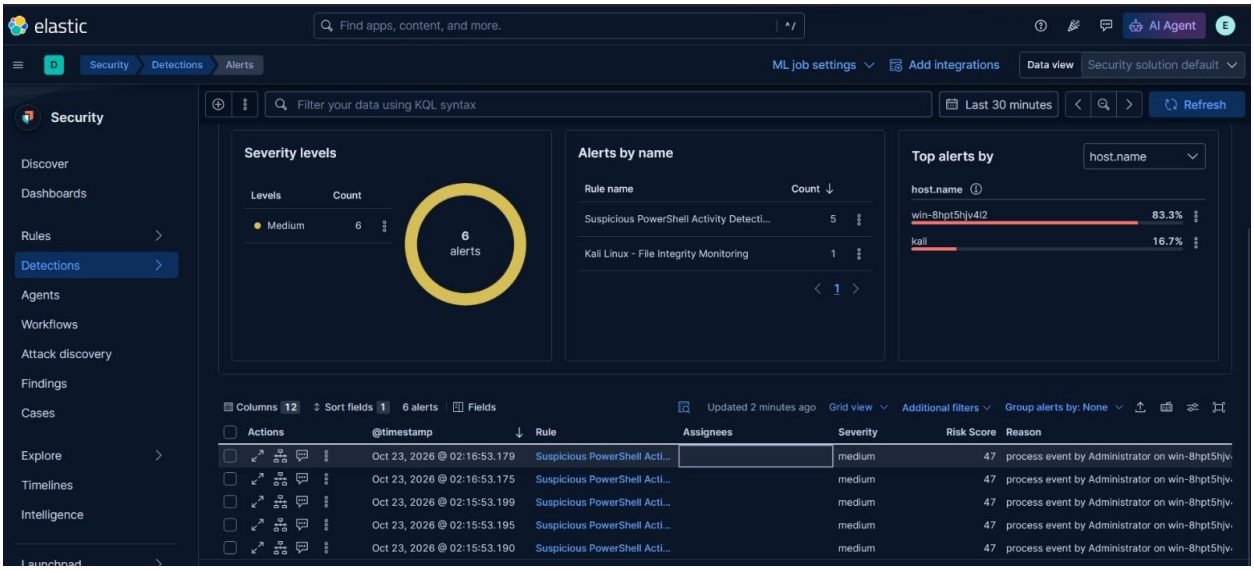


Figure 7 – Generated Suspicious PowerShell Detection Alerts

9. False Positive and False Negative Considerations

False Positives:

- Legitimate administrators may use Base64 decoding or PowerShell commands containing the monitored keywords.
- Approved administrative scripts may trigger the rule.

False Negatives:

- Attackers may use alternative encoding or obfuscation techniques.
- Malicious PowerShell activity may not contain the monitored keywords.

Tuning Recommendations:

- Correlate suspicious script content with process execution details.
- Exclude verified administrative scripts when appropriate.
- Monitor additional behavioral indicators rather than relying only on keywords.

10. Conclusion

The use case successfully demonstrated PowerShell Script Block Logging, centralized log collection, custom KQL detection engineering, and automated alert generation using Elastic Security. The five generated alerts validated the detection workflow for the controlled test scenario. Keyword detection identifies suspicious indicators and does not independently prove malicious execution.

SIEM Detection Engineering: Privilege Escalation Detection Engineering

Windows Local Group Membership Change Detection — SOC Engineering Report

1. Report Scenario

A controlled lab test was performed on Windows Server 2019 Datacenter to simulate a potential privilege escalation activity. A temporary user named FIM_TestUser was created and added to the local Administrators group. This action generated Windows Security Event ID 4732, which was collected and forwarded to Elastic Stack through Filebeat/Elastic Agent. A custom KQL detection rule was configured in Kibana to detect this activity and generate a high-severity alert.

2. Lab Environment

- **Platform:** Elastic Stack 9.x
- **Target OS:** Windows Server 2019 Datacenter
- **Log Source:** Windows Security Logs
- **SIEM Server:** Ubuntu
- **Detection Method:** Custom KQL-Based Detection
- **Event ID:** 4732
- **Test Account:** FIM_TestUser
- **Target Group:** Administrators
- **MITRE ATT&CK:** T1098.007 – Additional Local or Domain Groups

3. Commands and Detection Queries

The following PowerShell commands and KQL queries were utilized throughout the lab implementation and detection engineering workflow:

Inspecting the Administrators Group SID

```
Get-LocalGroup | Where-Object { $_.SID -eq "S-1-5-32-544" }
```

Purpose: Queries local security groups to verify the SID corresponding to the built-in local Administrators group (S-1-5-32-544).

Creating the Test Account

```
New-LocalUser -Name "FIM_TestUser" -Password (Read-Host -AsSecureString "Enter a test password") -Description "Temporary SIEM detection test account"
```

Purpose: Creates a new local user account with a secure password and descriptive metadata for detection testing.

Adding the User to the Administrators Group

```
Add-LocalGroupMember -Group "Administrators" -Member "FIM_TestUser"
```

Purpose: Assigns the newly created test user to the local Administrators group, simulating local privilege escalation.

Verifying the User SID

```
Get-LocalUser -Name "FIM_TestUser" | Select-Object Name, SID, Enabled
```

Purpose: Validates that the test account was created successfully, inspecting its assigned security identifier (SID) and enabled status.

KQL Detection Rule and Discover Query

```
event.code: "4732"
```

Purpose: Filters security telemetry in Kibana Discover and elastic security rules to identify Windows Security Event ID 4732 (member added to security-enabled local group).

4. Detection Rule Configuration

- **Rule Name:** Windows Local Group Membership Change Detection
- **Rule Type:** Custom Query
- **Severity:** High
- **Risk Score:** 73
- **Schedule:** Every 1 minute
- **Additional Look-back Time:** 1 minute
- **Data Stream:** system.security

5. Test Results and Alert Validation

The detection engineering and validation workflow confirmed the following findings:

- **Account Creation:** The test account FIM_TestUser was successfully created on the target Windows Server.
- **Group Assignment:** The account was successfully added to the local Administrators group.

- **Event Generation:** Windows Security Event ID 4732 was successfully generated by the operating system.
- **Kibana Discover Verification:** Kibana verified the incoming log event containing key ECS fields: event.code: 4732, event.action: added-member-to-group, and group.name: Administrators.
- **Alert Trigger:** Elastic Security evaluated the rule and successfully generated a High-severity alert with a Risk Score of 73.

6. Screenshot Arrangement and Explanations

The screenshots below document the chronological workflow from initial environment inspection to alert generation and verification.

1. Inspecting the Local Administrators Group and its SID

```
Get-LocalGroup | Where-Object { $_.SID -eq "S-1-5-32-544" }
```

PowerShell was used to inspect local groups and confirm that the target Administrators group corresponds to SID S-1-5-32-544. This ensures precise target scoping prior to simulation.

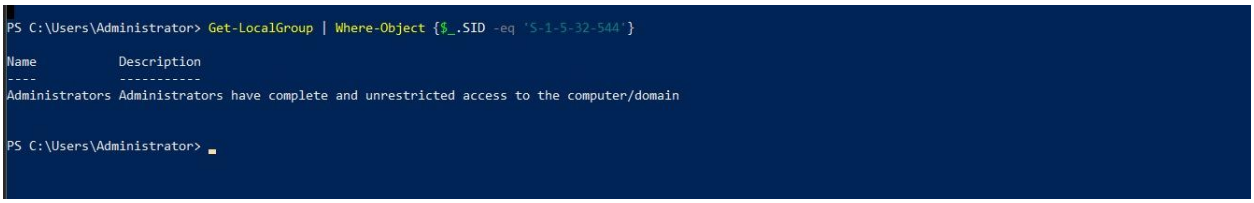


Figure 1: Inspecting the local Administrators group and verifying its SID via PowerShell.

2. Creating the Temporary Test Account

```
New-LocalUser -Name "FIM_TestUser" -Password (Read-Host -AsSecureString "Enter a test
password") -Description "Temporary SIEM detection test account"
```

A temporary local user account named FIM_TestUser was created with a secure password and descriptive metadata. This establishes the baseline object required for privilege escalation simulation.



Figure 2: Creating the temporary test account FIM_TestUser on Windows Server 2019.

3. Adding FIM_TestUser to the Administrators Group

```
Add-LocalGroupMember -Group "Administrators" -Member "FIM_TestUser"
```

The test user was added to the local Administrators group to simulate unauthorized privilege escalation. This administrative modification immediately triggers Windows security auditing.

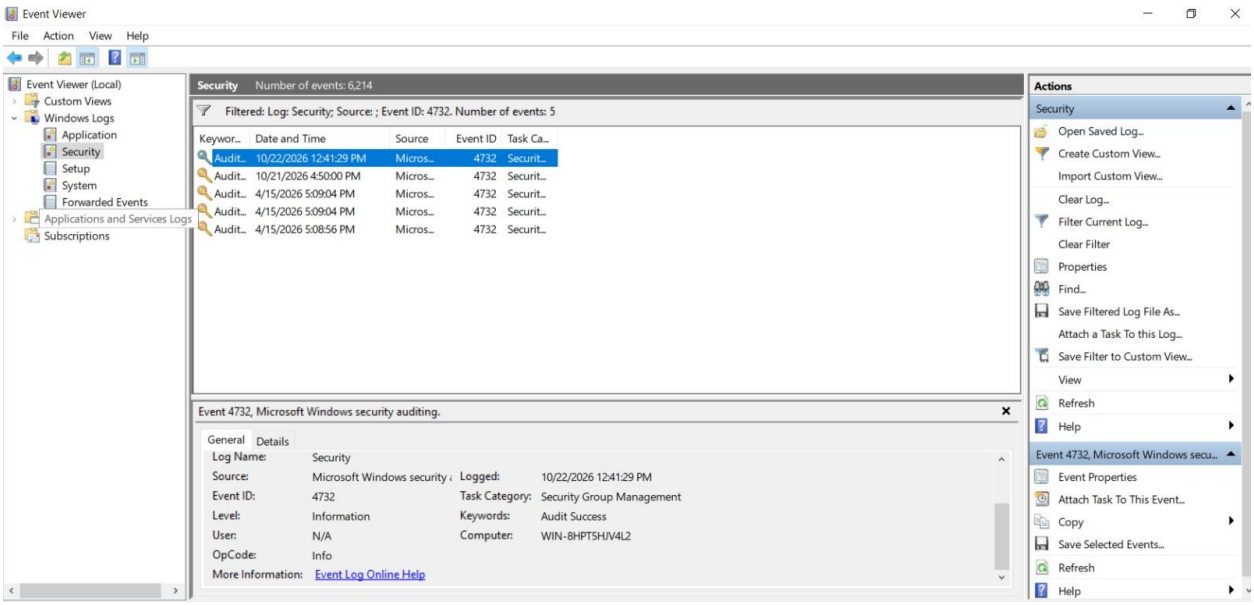


Figure 3: Event Viewer showing Windows Security Event ID 4732 generated upon group membership modification.

4. Verifying the Account SID and Status

```
Get-LocalUser -Name "FIM_TestUser" | Select-Object Name, SID, Enabled
```

PowerShell verification confirmed the account's assigned SID and verified that the user account is enabled. This confirms proper local object creation before SIEM ingestion.

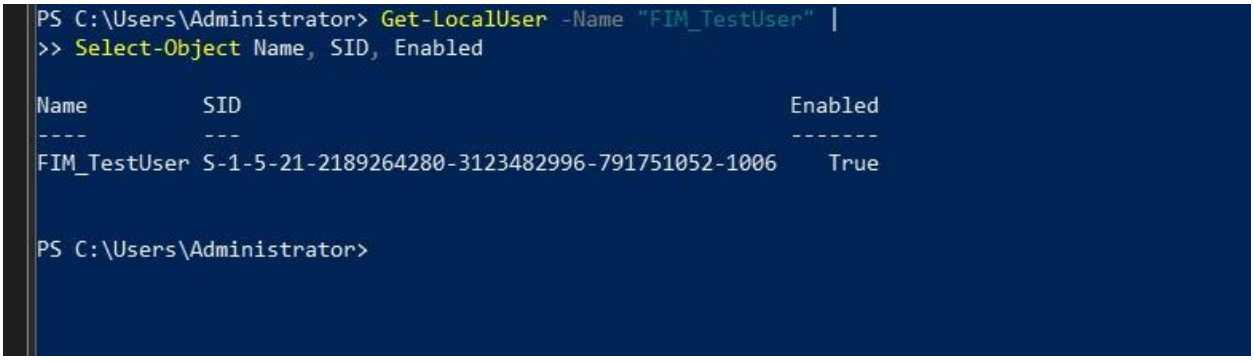


Figure 4: Verifying the test user account name, SID, and enabled status via PowerShell.

5. Searching for Event ID 4732 in Kibana Discover

```
event.code: "4732"
```

Kibana Discover was queried using event.code: "4732" to verify that log telemetry from Windows Server 2019 was successfully ingested. The resulting document confirms successful log pipeline transport via Elastic Agent.

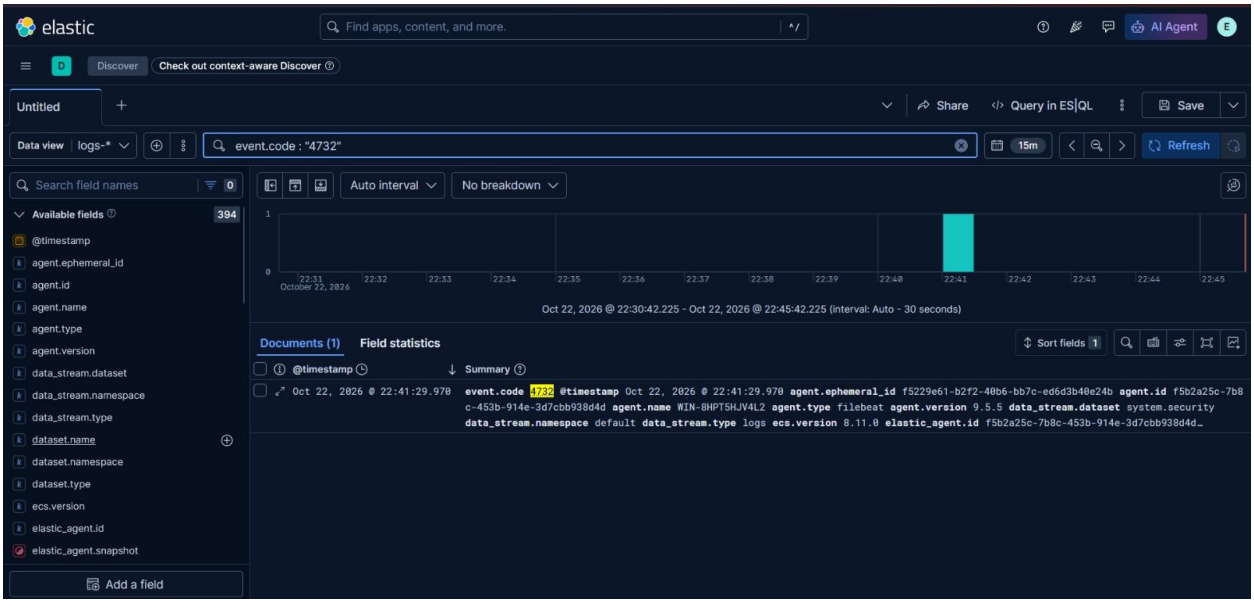


Figure 5: Kibana Discover interface displaying ingested event telemetry for Event ID 4732.

7. Configuring the KQL Detection Rule

```
event.code: "4732"
```

A custom KQL detection rule was configured in Elastic Security with High severity and a risk score of 73. The rule is scheduled to run every 1 minute with a 1-minute look-back window against system.security data streams.

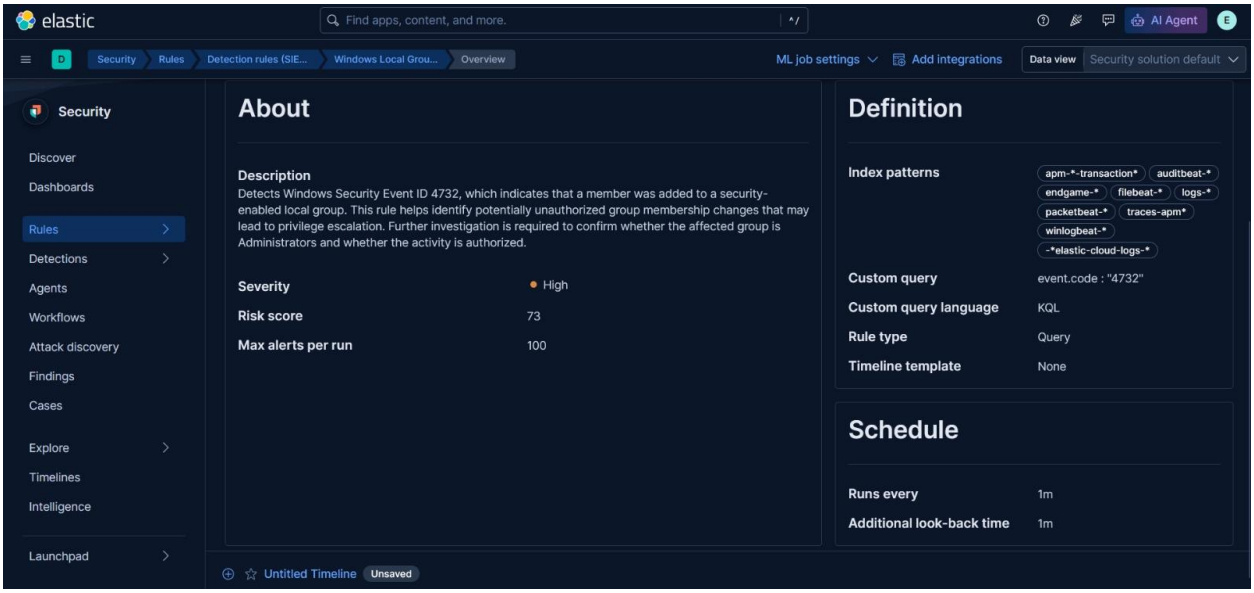


Figure 7: Detection rule configuration settings and KQL query definition in Elastic Security.

8. Showing the Saved and Enabled Detection Rule

```
event.code: "4732"
```

The detection rule was successfully saved, enabled, and verified within the Elastic Security Rules management dashboard. Periodic execution checks confirm continuous rule health and successful recent runs.

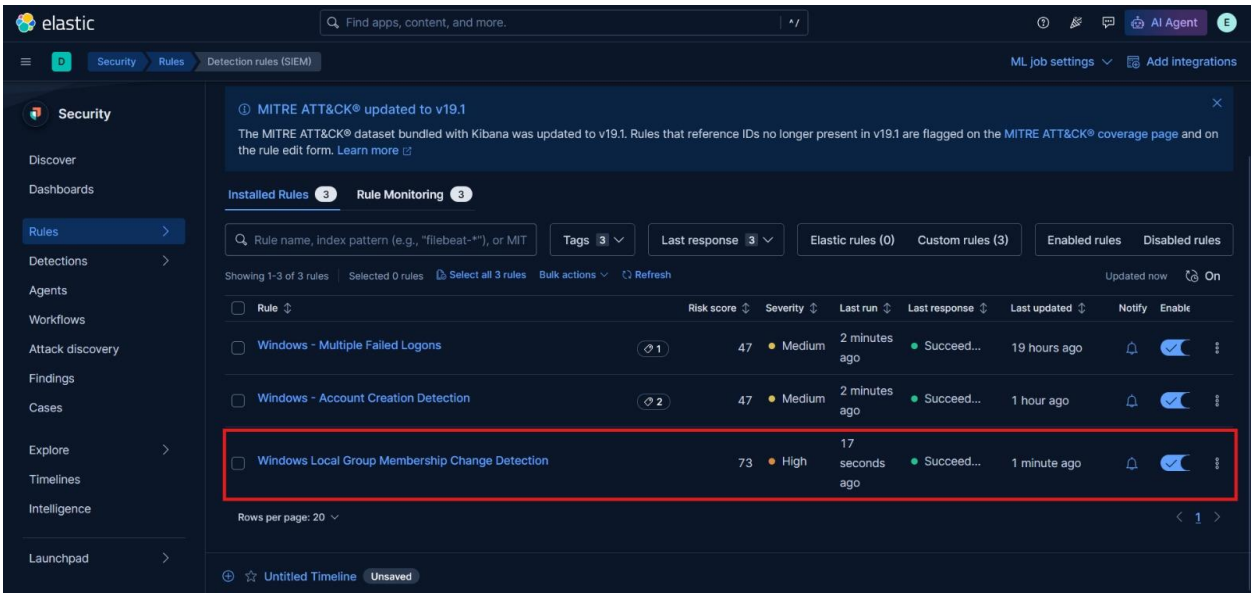


Figure 3 (Dashboard): Saved and enabled detection rules list showing active operational status.

9. Displaying the Generated High-Severity Alert

```
event.code: "4732"
```

Elastic Security successfully evaluated the incoming telemetry against the configured rule and generated a High-severity alert. The alert correctly captures the risk score, timestamp, and rule metadata for SOC triage.

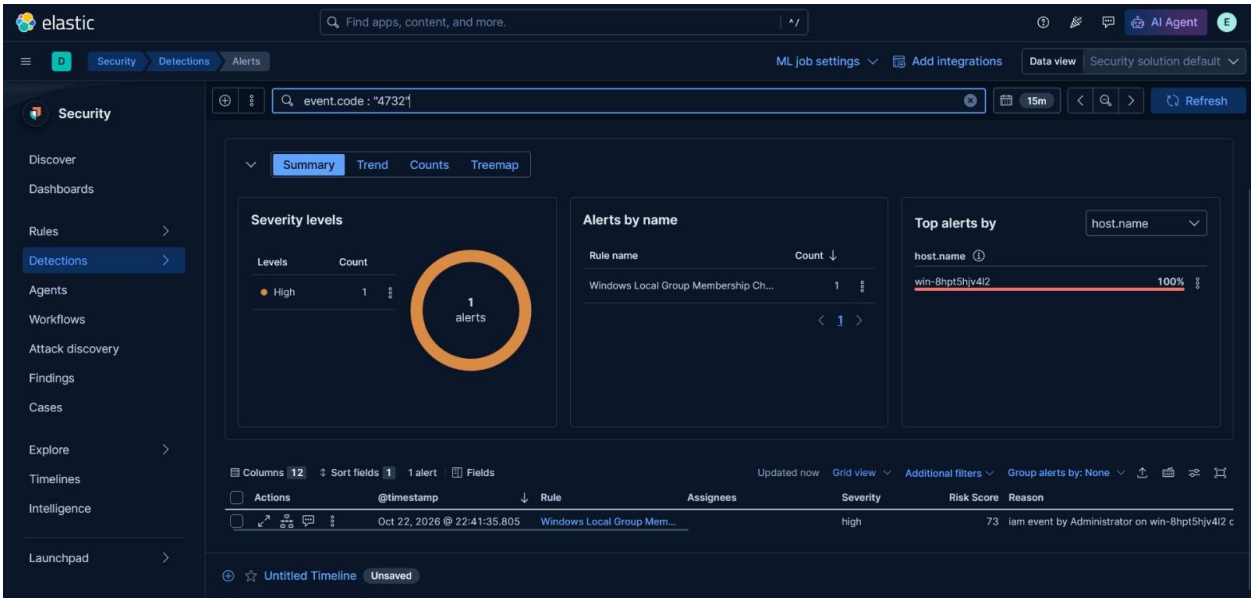


Figure 9: Security Detections dashboard displaying the generated High-severity alert.

7. Conclusion

The detection rule successfully identified a local Administrators group membership change through Windows Security Event ID 4732 and generated a High-severity alert with a risk score of 73. Clarify that this was an authorized lab test, and that the event indicates a potentially suspicious privilege change but does not by itself prove malicious activity.

SIEM Detection Engineering: File Integrity Monitoring (FIM) on Kali Linux Using Elastic Stack

1. Report Scenario

A controlled lab environment was established to implement File Integrity Monitoring (FIM) on a Kali Linux system using Elastic Stack 9.5.5. The primary objective is to detect unauthorized or suspicious file creations, modifications, and attribute changes within critical system and test directories. File integrity events are collected via Elastic Agent and analyzed in Elastic Security to generate real-time security alerts.

2. Lab Environment

- **SIEM Platform:** Elastic Stack 9.5.5
- **Operating System:** Kali Linux
- **Monitoring Agent:** Elastic Agent 9.5.5
- **Integration:** FIM-SOC
- **Agent Policy:** Kali-Linux-SOC-Policy
- **Event Dataset:** fim.event
- **Test Directory:** /home/kali/FIM-Test/

3. FIM Integration Configuration

File Integrity Monitoring was configured within Fleet using the FIM integration to continuously monitor critical system paths and dedicated test directories. The monitoring scope includes system binary directories, configuration paths, and user test spaces to ensure comprehensive visibility over file modifications.

- **Monitored Path:** /bin
- **Monitored Path:** /usr/bin
- **Monitored Path:** /sbin
- **Monitored Path:** /usr/sbin
- **Monitored Path:** /etc
- **Monitored Path:** /home/kali/FIM-Test

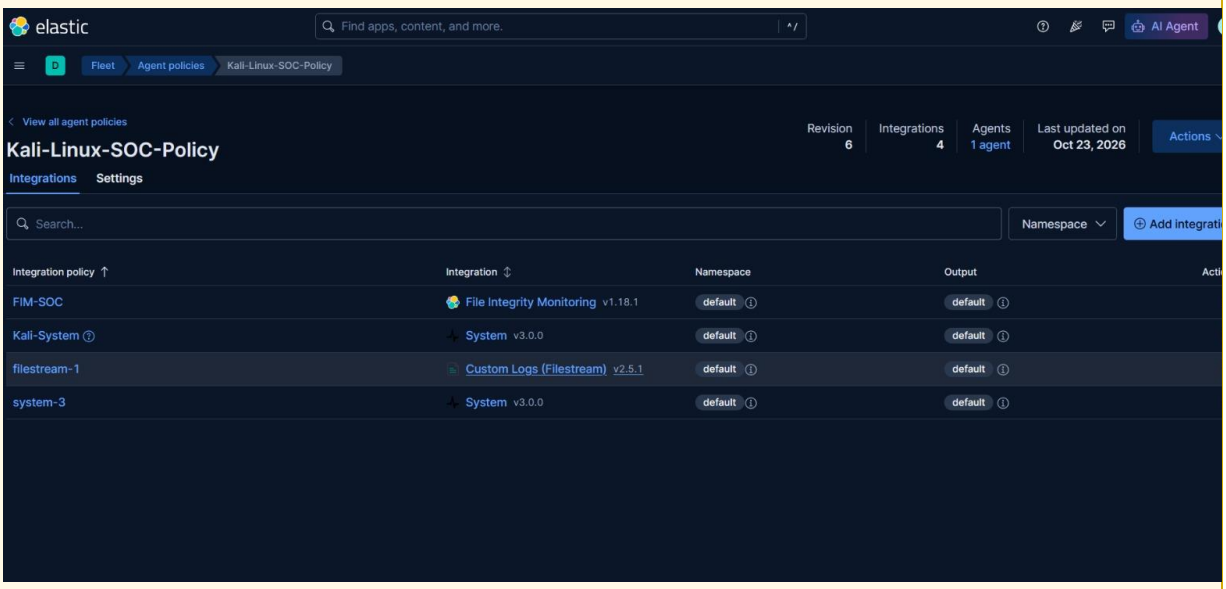


Figure 1 – FIM Integration Configuration in Fleet.

4. Test Directory and File Creation

A dedicated test directory and baseline file were created on Kali Linux to simulate file modifications and verify FIM event ingestion.

Creating the Test Directory

```
mkdir -p /home/kali/FIM-Test
```

Purpose: Creates the dedicated testing directory structure for file integrity validation.

Creating the Initial Test File

```
echo "FIM initial test" > /home/kali/FIM-Test/test.txt
```

Purpose: Generates a baseline text file within the test directory to establish initial file state tracking.

Correcting File and Directory Ownership

```
sudo chown kali /home/kali/FIM-Test
sudo chown kali /home/kali/FIM-Test/test.txt
```

Purpose: Ensures proper permissions and user ownership for testing operations.

Modifying the Test File

```
echo "FIM modification test" >> /home/kali/FIM-Test/test.txt
```

Purpose: Appends text to the test file to trigger file modification event generation.

5. Kibana Discover – FIM Event Verification

Collected file integrity events were verified in Kibana Discover to confirm that file modifications on Kali Linux were successfully ingested by the SIEM pipeline. The analysis focused on verifying core ECS (Elastic Common Schema) event fields.

Kibana Discover Query

```
data_stream.dataset: "fim.event" and file.path: "/home/kali/FIM-Test/test.txt"
```

Purpose: Filters telemetry in Kibana Discover to isolate file integrity events corresponding to the test file.

Verified Event Fields:

- **event.action:** updated, attributes_modified
- **event.category:** file
- **event.type:** change
- **file.path:** /home/kali/FIM-Test/test.txt
- **host.name:** kali
- **file.hash.sha1:** Cryptographic hash tracking for file integrity validation

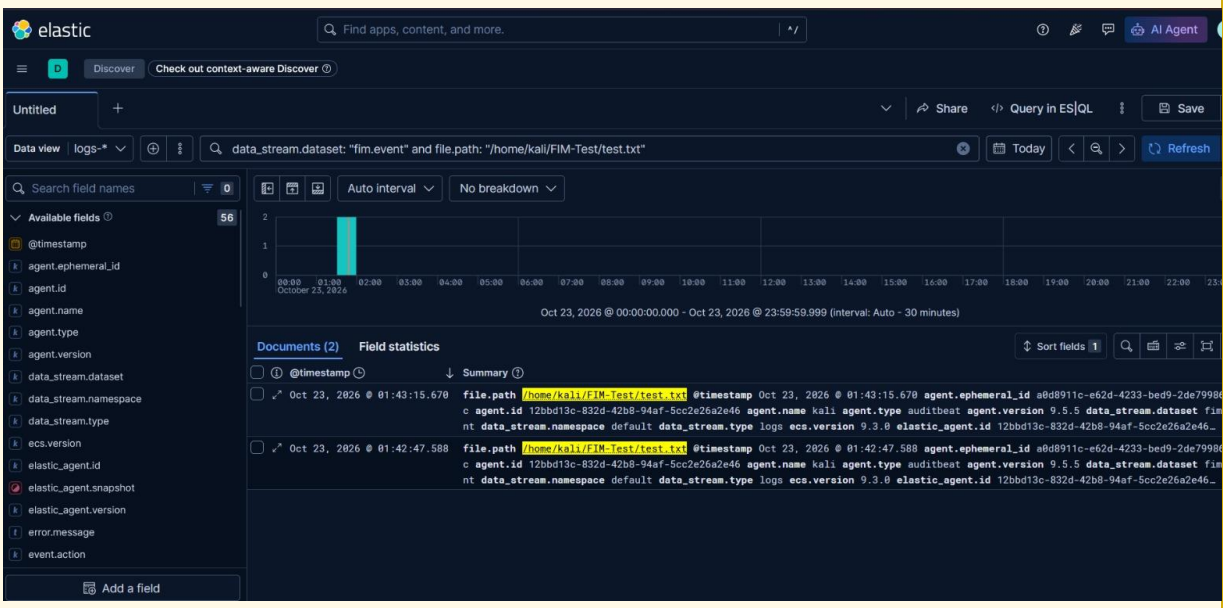


Figure 3 – Kibana Discover interface showing verified fim.

6. Custom Detection Rule Configuration

A custom detection rule was created in Elastic Security to automatically detect file integrity changes on the Kali Linux host and trigger security alerts.

Custom Detection Query

```
data_stream.dataset: "fim.event" and
host.name: "kali" and
event.category: "file" and
event.type: "change"
```

Purpose: Evaluates incoming log streams to identify file modification events specifically originating from the Kali Linux host.

- **Rule Name:** Kali Linux - File Integrity Monitoring
- **Rule Type:** Custom Query
- **Severity:** Medium
- **Risk Score:** 47

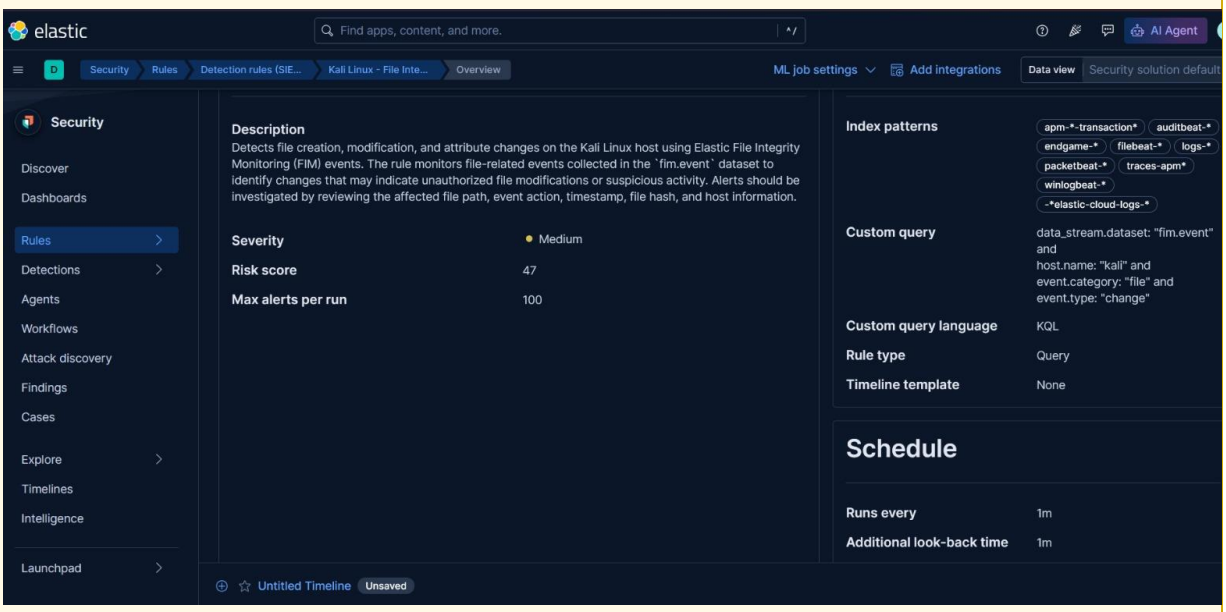


Figure 4 – Custom detection rule configuration interface in Elastic Security showing rule name, KQL query, severity, and risk score.]

7. Detection Rule Activation and Testing

After enabling the detection rule in Elastic Security, active testing was conducted by performing subsequent file modifications and file creations to validate real-time alert generation.

File Modification Test

```
echo "FIM detection rule test $(date -Is)" >> /home/kali/FIM-Test/test.txt
```

Purpose: Generates a time-stamped file modification event to test rule responsiveness.

File Creation Test

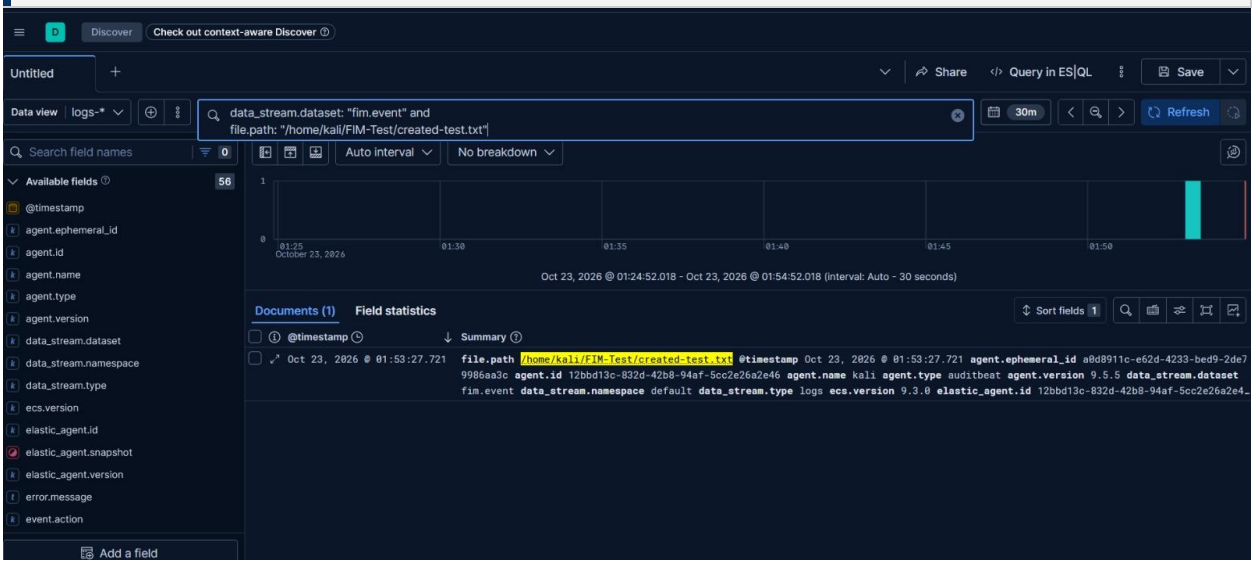
```
echo "FIM file creation test $(date -Is)" > /home/kali/FIM-Test/created-test.txt
```



Purpose: Creates a new file in the monitored directory to verify detection capabilities.

Verification Query

```
data_stream.dataset: "fim.event" and
file.path: "/home/kali/FIM-Test/created-test.txt"
```



Purpose: Verifies ingestion of the newly created test file event.

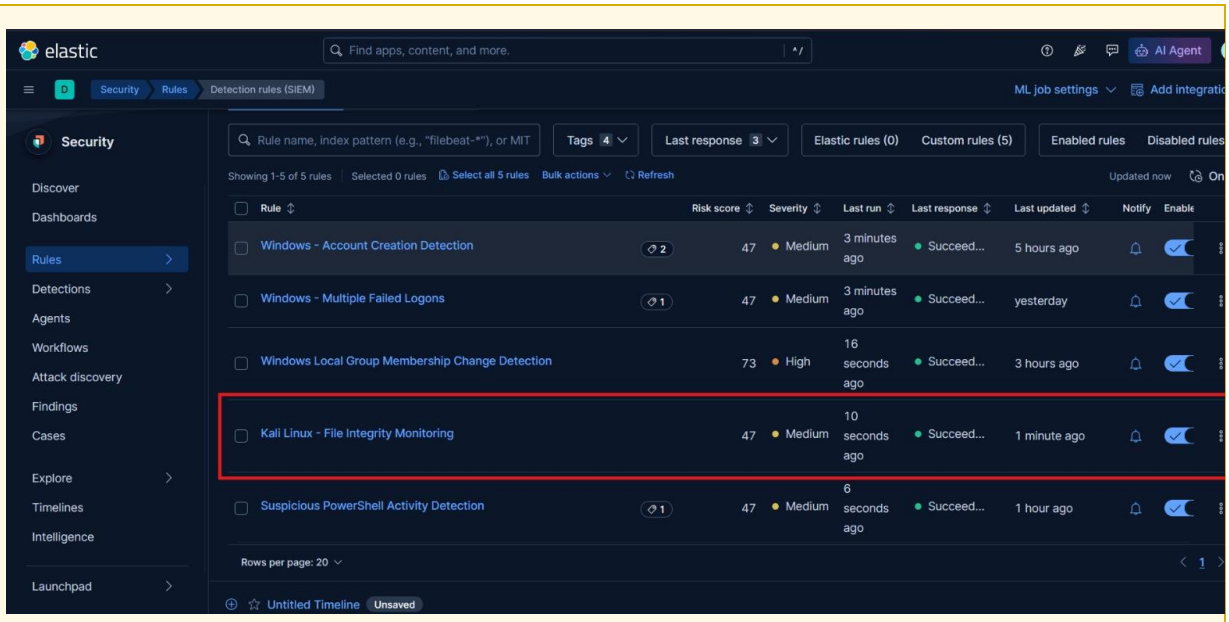


Figure 5 – Terminal session and Elastic Security rules dashboard showing rule activation and subsequent live testing commands.]

8. Detection Results and Alerts

Elastic Security successfully evaluated the incoming file integrity telemetry against the configured rule, automatically generating security alerts upon detecting file modifications.

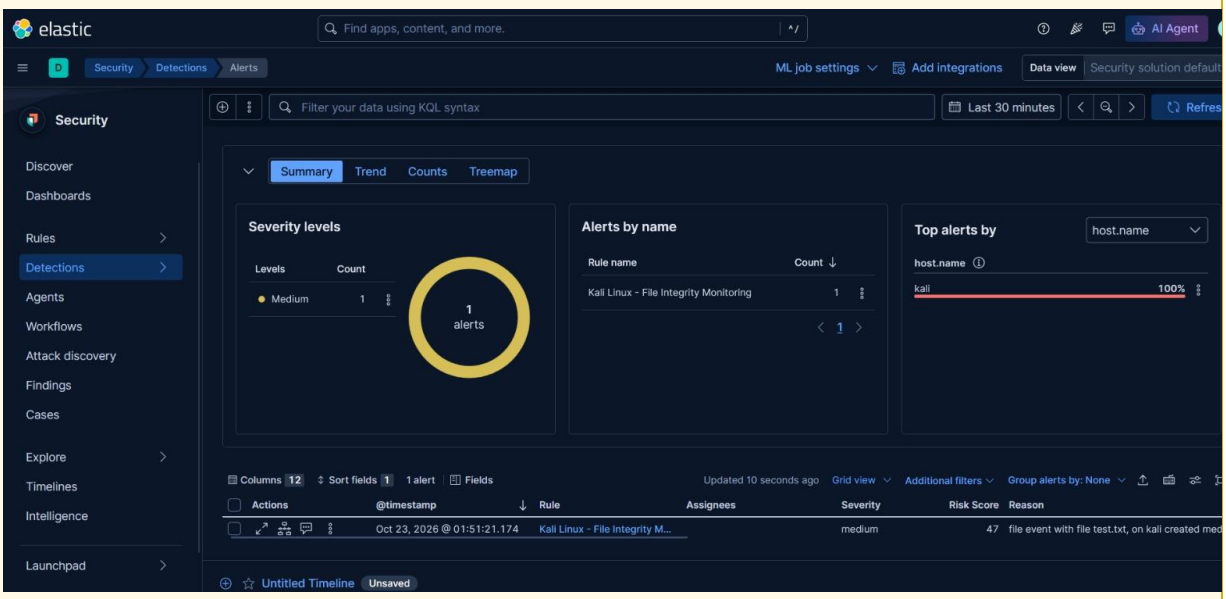


Figure 6 – Elastic Security Detections dashboard displaying generated Medium-severity FIM alerts with risk score 47 and event metadata.]

9. False Positive and False Negative Considerations

To maintain effective SOC operations, potential detection limitations and tuning strategies were evaluated:

False Positives

- **Administrative Changes:** Legitimate system administration activities and configuration updates may trigger unintended alerts.
- **Software Updates:** Authorized package management updates and system patching generate widespread file integrity events.

False Negatives

- **Unmonitored Paths:** Modifications occurring outside configured monitoring directories will not be captured.
- **Agent Failures:** Monitoring service or Elastic Agent interruptions can result in missing telemetry and undetected changes.

Tuning Recommendations

- **Monitor Critical Paths:** Focus monitoring scope on sensitive system binaries and configuration directories.
- **Investigate Unexpected Changes:** Establish triage playbooks for unauthorized file modifications in protected paths.
- **Exclude Trusted Activities:** Apply precise exception filters for authorized maintenance windows and verified software deployments.

10. Conclusion

The project successfully implemented File Integrity Monitoring (FIM) on Kali Linux using Elastic Stack 9.5.5. Through comprehensive event collection, ECS verification, custom KQL detection engineering, and alert validation, the SIEM successfully detected unauthorized file modifications and generated automated security alerts suitable for university project submission.

SIEM Detection Engineering: Windows Account Creation

Elastic SIEM – Detection Engineering Lab

1. Report Scenario

The objective of this use case is to detect the creation of new local user accounts on Windows Server 2019 using Elastic SIEM and Windows Security Events. The detection maps to MITRE ATT&CK technique T1136.001 (Create Account: Local Account).

2. Environment

- **SIEM Platform:** Elastic Stack 9.x
- **Operating System:** Windows Server 2019 Datacenter (host: WIN-8HPT5HJV4L2)
- **Log Collection:** Filebeat 9.5.5 (Elastic Agent)
- **Data Stream:** `logs-system.security-default`
- **Detection Type:** Custom Query (KQL)

3. Windows Auditing Configuration

The Windows User Account Management audit policy was checked and confirmed to be configured for successful events. Windows Security Event ID **4720** (a user account was created) was used to identify user account creation activity.

The policy can be verified from an elevated prompt with:

```
auditpol /get /subcategory:"User Account Management"
```

4. Log Collection and Verification

Windows Security events were successfully received and indexed in Elastic. The relevant events were verified in Kibana Discover (data view `logs-*`) using the following KQL query:

```
event.code : "4720"
```

The query returned Event ID 4720 documents collected by Filebeat from the Windows host, each containing the event code, the source agent, the data stream (`system.security`), and the event timestamp.

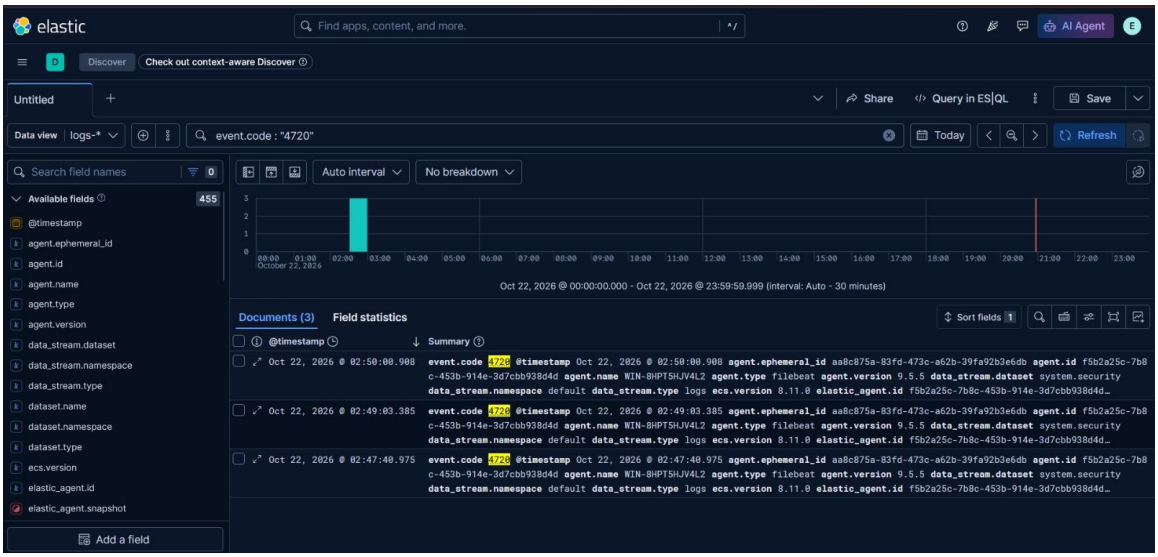


Figure 2 – Kibana Discover showing Windows Security Event ID 4720.

5. Detection Rule Configuration

A custom detection rule named **Windows - Account Creation Detection** was created and enabled in Elastic Security. The rule description reads: "Detects successful creation of Windows user accounts using Security Event ID 4720."

Detection Query:

```
event.code : "4720" and event.action : "added-user-account"
```

Rule Configuration:

- **Severity:** Medium
- **Risk Score:** 47
- **Execution Interval:** Every 5 minutes
- **Additional Look-back Time:** 1 minute
- **Max alerts per run:** 100
- **Tags:** `windows`, `account-creation`

The rule searches Windows Security events for successful user account creation activity. The Detection rules page confirmed that the rule was enabled and its last run **Succeeded**.

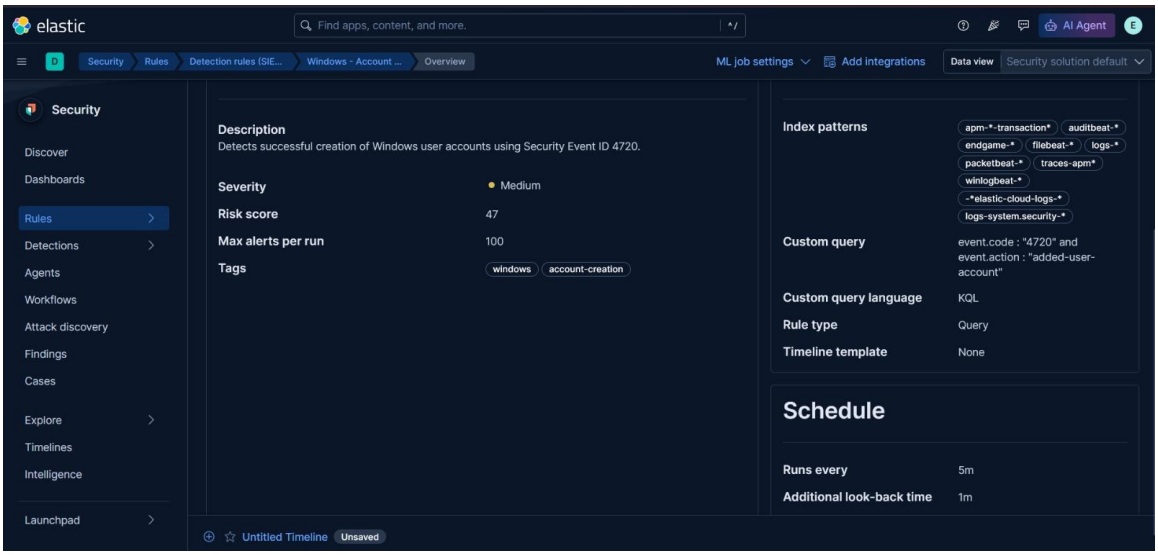


Figure 3 – Detection rule configuration (name, severity, risk score, tags, query and schedule).

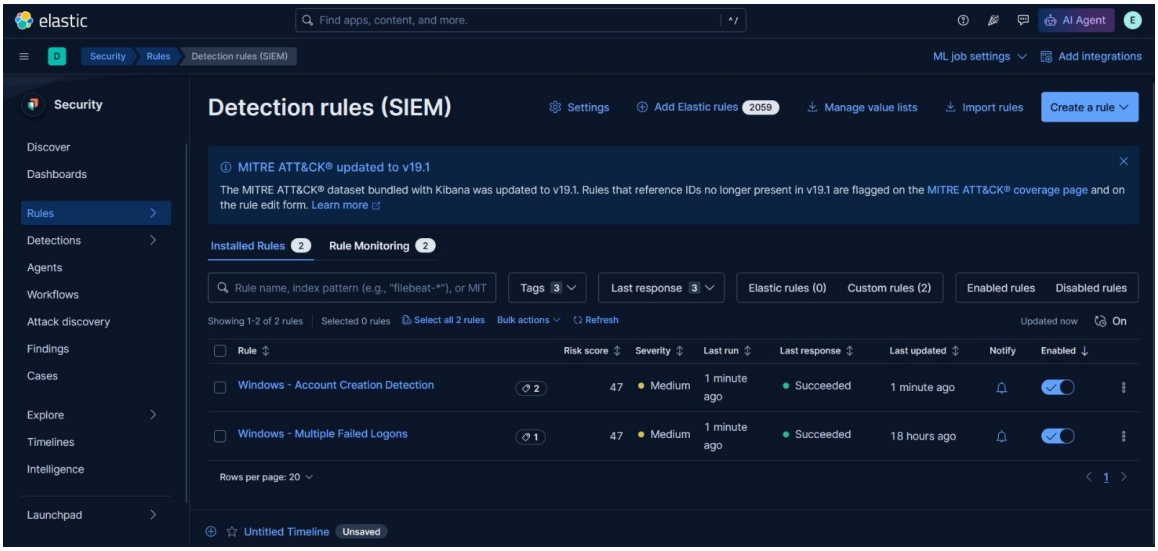


Figure 4 – Detection rules page: the custom rule is enabled and executing successfully.

6. Validation and Testing

6.1 Test Account Creation

After the detection rule was enabled, a test account named `acct_test03` was created on Windows Server 2019 using PowerShell:

```
New-LocalUser -Name "acct_test03" -NoPassword -Description "Elastic account creation detection test"
```

The command completed successfully and the account was created in an enabled state.

```
PS C:\Users\Administrator> New-LocalUser -Name "acct_test03" -NoPassword -Description "Elastic account creation detection test"

Name        Enabled Description
----        ------- -----------
acct_test03 True    Elastic account creation detection test


PS C:\Users\Administrator>
```

Figure 5 – Creation of the test account `acct_test03`.

6.2 Event Verification

The corresponding Event ID 4720 was located in Kibana Discover using the query below, confirming that the account creation event was collected successfully.

```
event.code : "4720" and user.target.name : "acct_test03"
```

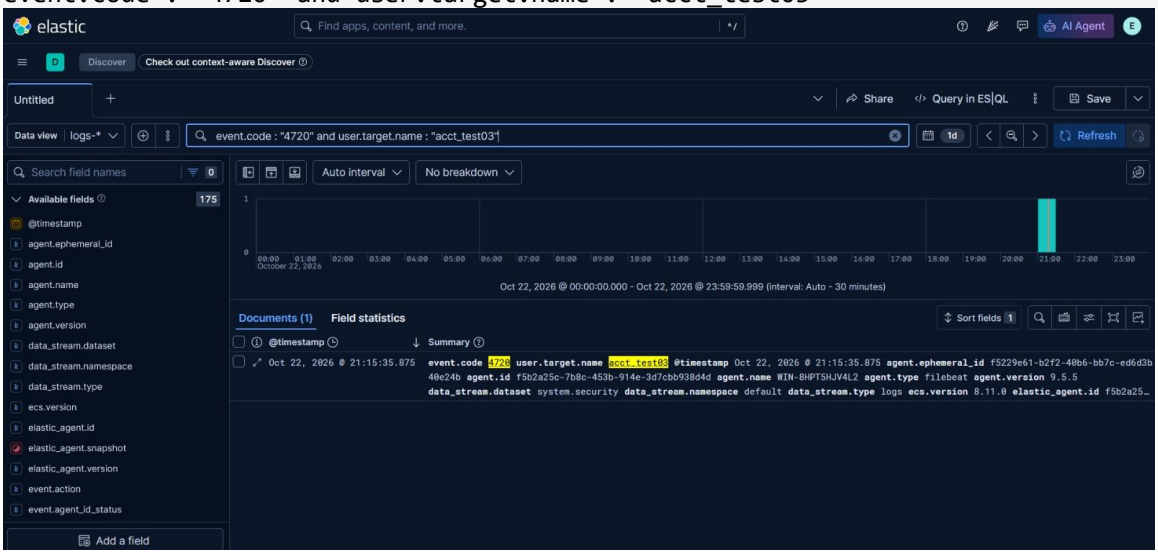


Figure 6 – Event ID 4720 for the target account `acct_test03`.

6.3 Alert Generation

An alert was subsequently generated by the **Windows - Account Creation Detection** rule. The alert appeared in Elastic Security with **Medium** severity and a risk score of 47, on host `win-8hpt5hjv4l2`.

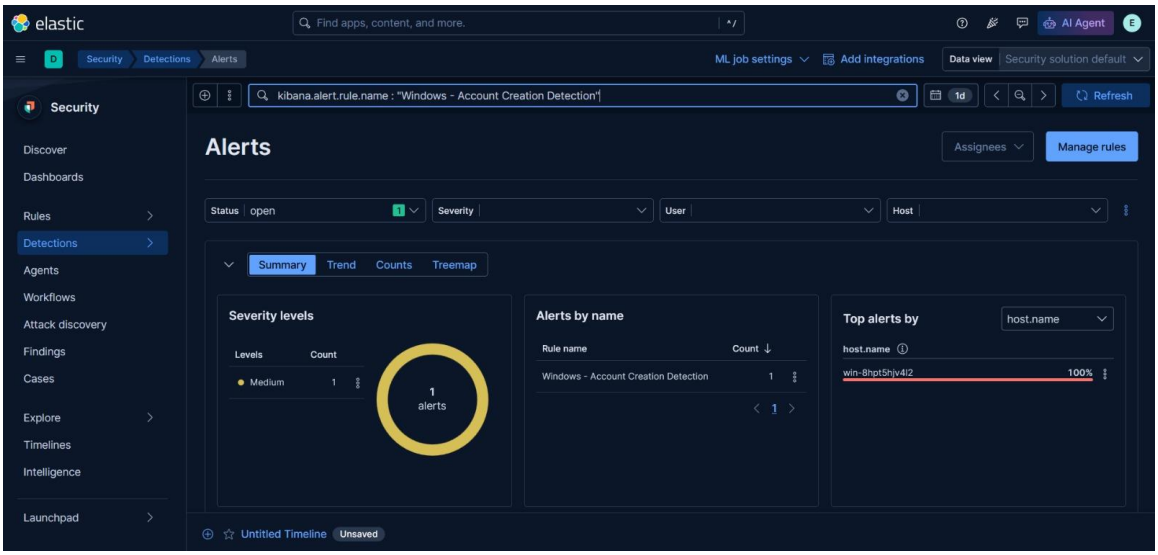


Figure 7 – Generated alert in Elastic Security.

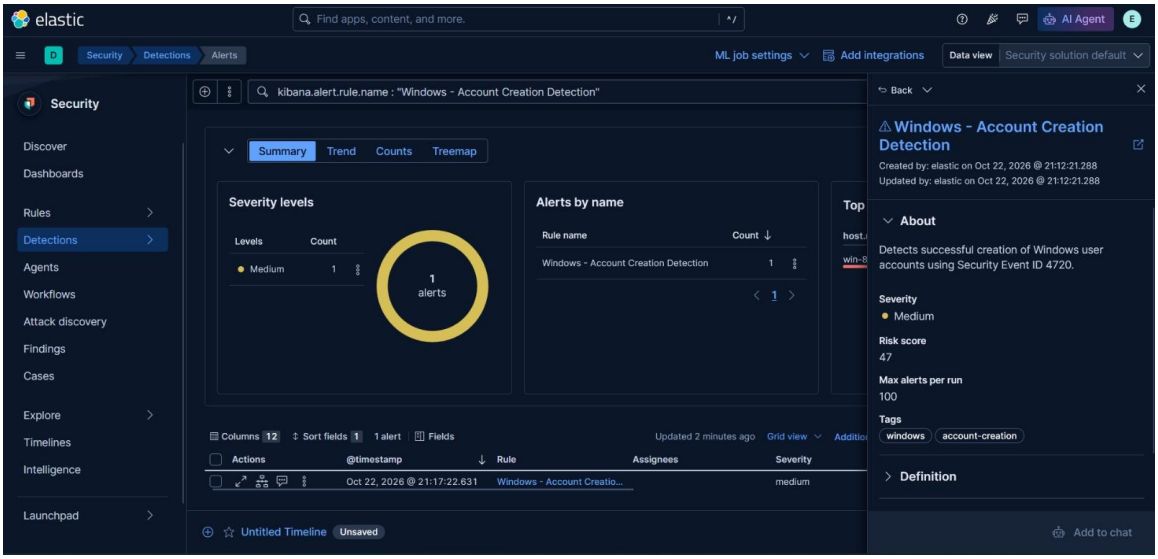


Figure 8 – Alert details for the Account Creation Detection rule.

6.4 Timeline

Time (Oct 22, 2026)	Activity
21:12:21	Detection rule created and enabled by `elastic`
21:15:35	Event ID 4720 logged for `acct_test03`
21:17:22	Alert generated by the rule

The alert was generated about 1 minute 47 seconds after the event was logged, which is within the 5-minute execution interval of the rule.

7. Result

The use case was successfully validated in the lab environment. Windows account creation activity was logged, collected by Elastic, and detected by the custom detection rule. The generated alert confirmed that the configured detection logic operated as expected during the test.

8. Conclusion

This use case demonstrates how Elastic SIEM can monitor Windows account creation activity and generate alerts for investigation. It provides a foundation for detecting potentially unauthorized account creation in a Windows environment.

9. Possible Improvements

- Exclude known, authorized administrative accounts or approved change windows to reduce false positives.
- Correlate with Event ID 4732 (member added to a security-enabled local group) to catch new accounts added to the Administrators group.
- Raise severity when the account is created outside business hours or by a non-administrative user.

Note: Account creation is not inherently malicious. Alerts should be investigated in context to determine whether the activity was authorized.

SOC DETECTION ENGINEERING | SIEM USE CASE DOCUMENTATION

UC07 – Remote Security Log Tampering

Implementation, attack simulation, detection logic and validation using an Elastic EQL sequence rule

Project	Project 4: SIEM Detection Engineering, Alert Triage & SOC Operations
Program	Digital Egypt Pioneers Initiative
Use case	UC07 – Remote Security Log Tampering
Severity / Risk score	Critical / 90
SIEM platform	Elastic Security 8.19.23
Detection type	EQL sequence rule (Event Correlation)
Test date	November 2, 2026
Prepared by	SOC Lab Team
Document status	Version 1.0. Screenshots listed as pending in Section 10 still need to be added.

Table of Contents

1. Document Objective 3

1.1 How to read this document 3

1.2 Evidence sources 3

2. Lab Environment 3

2.1 Notes on the environment 4

3. Attack Simulation – Step by Step 4

3.1 Prerequisites for reproduction 4

3.2 Step 1: Establish Remote PowerShell Access 4

3.3 Step 2: Modify the Windows Audit Policy 5

3.4 Step 3: Clear the Windows Security Event Log 6

4. Detection and Event Correlation 7

4.1 Detection workflow 7

4.2 The three-event detection sequence 7

4.3 Evidence from the successful test 7

5. Elastic EQL Detection Rule 9

5.1 Rule summary 9

5.2 Final EQL query 10

5.3 Query explained 10

5.4 Important technical note on Logon ID 10

5.5 Possible hardening (not implemented, not tested) 11

6. Rule Scheduling Configuration 11

6.1 Purpose of periodic execution and look-back 12

7. Detection Results and Validation 12

7.1 Alert generated 12

7.2 What the alert confirms 13

7.3 Why correlation is better than detecting Event ID 1102 alone 13

7.4 Value for a SOC analyst 13

7.5 New rule versus the older rule 13

8. MITRE ATT&CK Mapping 14

9. SOC Analyst Investigation and Response 15

9.1 Investigation procedure 15

9.2 Useful Discover queries 15

9.3 Containment and response 15

9.4 Recommended defensive measures 16

10. Screenshots and Evidence 16

10.1 Evidence register 16

10.2 Open items and integrity notes 16

11. Conclusion 17

1. Document Objective

This document records the implementation, simulation, detection logic and validation of use case UC07 – Remote Security Log Tampering in the Elastic SIEM lab. It follows the attack simulation step by step: remote command-line access to a Windows Server 2019 over WinRM, modification of the Windows audit policy, and clearing of the Security event log. It ends with the detection of that sequence by an Elastic EQL detection rule.

The document covers:

- The lab environment and the exact commands that were executed.
- The Windows events produced by the simulation (Event IDs 4624, 4719 and 1102) and how they are correlated.
- The final EQL rule, its schedule and the alert it generated.
- A MITRE ATT&CK mapping that claims only what the evidence supports.
- A SOC investigation and response procedure, and recommended defensive measures.

1.1 How to read this document

Observed facts and analyst reasoning are kept apart. Three labels are used:

> **Observed evidence:** what is visible in a screenshot or recorded in the lab data supplied by the team.

> **Interpretation:** the analyst's reading of that evidence. It is reasoning, not an additional observation.

> **Evidence pending:** a screenshot that is required by the use case but has not been provided yet. A labeled placeholder marks its position.

1.2 Evidence sources

- **Screenshots provided:** the remote PowerShell session (Figure 1), the rule definition and schedule pages (Figures 7 and 8) and the Alerts page (Figure 9).
- **Lab records supplied by the team:** hostnames, IP addresses, Logon ID and the event timeline in Section 4. These were taken from Kibana Discover during the test. The Discover screenshots (Figures 3 to 6) are pending, so these values are not yet backed by an image in this document.

2. Lab Environment

Component	Details
SIEM platform	Elastic Security 8.19.23
Source machine	Windows 11
Source hostname	SOFTLAPTOP
Source IP	192.168.1.175
Target machine	Windows Server 2019
Target hostname	WIN-MKSP5LUC3TN
Target IP	192.168.1.116
Target domain	test.local
Remote access protocol	WinRM
Remote access port	TCP 5985

Component	Details
Log integration	System Integration
Data stream	logs-system.security-*
Detection type	EQL Sequence Rule

2.1 Notes on the environment

- The Security log of the target is collected by Elastic Agent (managed by Fleet) with the System integration and stored in the `logs-system.security-*` data streams.
- Figure 1 confirms the target address (192.168.1.116) and the account used (TEST\Administrator). The source hostname and IP are not visible in that screenshot and come from the lab records.
- All activity was performed in an isolated lab. The simulation clears the Security log of the target and must not be repeated on a production system.

3. Attack Simulation – Step by Step

The simulation reproduces a common defense-evasion pattern: an attacker with administrator credentials connects remotely, weakens the audit policy and then clears the Security log to remove traces. The three steps were run in one remote PowerShell session.

3.1 Prerequisites for reproduction

These are general prerequisites for another analyst repeating the test. They are not all shown in the screenshots.

- An isolated lab with a Windows Server 2019 target and a separate source machine on the same network.
- WinRM enabled on the target and TCP 5985 reachable from the source (for example, `Test-NetConnection 192.168.1.116 -Port 5985`).
- Credentials of an administrator account on the target (`TEST\Administrator` in this lab).
- Elastic Agent healthy on the target, the System integration collecting the Security log, and the detection rule enabled before the test starts.
- A note of the start time of each step, so the events can be matched later.

3.2 Step 1: Establish Remote PowerShell Access

From the Windows 11 source machine, a remote PowerShell session was opened to the Windows Server 2019 target:

```
$cred = Get-Credential -UserName "TEST\Administrator"
Enter-PSSession -ComputerName 192.168.1.116 -Credential $cred
```

> **Observed evidence:** Figure 1 shows the Get-Credential prompt for TEST\Administrator, followed by Enter-PSSession to 192.168.1.116. The prompt then changes to `[192.168.1.116]: PS C:\Users\Administrator\Documents>`. A `CreateRemoteRunspaceFailed` error is visible at the top of the capture; its cause is not shown.

What PowerShell Remoting is. PowerShell Remoting lets an administrator run PowerShell commands on another computer. `Enter-PSSession` starts an interactive one-to-one session: each

command is typed locally, executed on the target with the rights of the account used, and the output is returned to the console.

How WinRM enables remote administration. PowerShell Remoting runs on Windows Remote Management (WinRM), Microsoft's implementation of the WS-Management protocol. The WinRM service on the target listens for requests, authenticates the caller (Kerberos or NTLM) and starts a remote PowerShell runspace, hosted in a `wsmprovhost.exe` process, to execute the commands.

Why TCP 5985 is used. 5985 is the default port of the WinRM HTTP listener (5986 is used for HTTPS). Over HTTP, WinRM still encrypts the message payload when Kerberos or NTLM authentication is used, but the transport is not protected by TLS and the server is not authenticated by a certificate. HTTPS on 5986 is preferred in production.

Security implications. Remote administrative access gives the caller the full rights of the account. It is legitimate for administrators, and it is also a common way for attackers to move laterally and work interactively on a host. The main controls are:

- Allow TCP 5985/5986 only from approved management hosts, and prefer HTTPS.
- Limit which accounts may log on over the network, and avoid using the built-in Administrator account for routine remote work.
- Use Just Enough Administration (JEA), multi-factor authentication or a privileged access workstation where possible.
- Monitor network logons (Event ID 4624, Logon Type 3) by administrative accounts, and remote-session process activity such as `wsmprovhost.exe`.

3.3 Step 2: Modify the Windows Audit Policy

Inside the remote session, the failure-auditing setting of the "Other Policy Change Events" subcategory was disabled and then enabled again:

```
auditpol /set /subcategory:"Other Policy Change Events" /failure:disable

auditpol /set /subcategory:"Other Policy Change Events" /failure:enable
```

> **Observed evidence:** Figure 1 shows both commands. Each returned "The command was successfully executed."

What the Windows audit policy is. The audit policy defines which security-relevant actions Windows writes to the Security log. The Advanced Audit Policy is organized in categories and subcategories, and each subcategory can audit Success events, Failure events or both. It is managed with `auditpol.exe` or Group Policy.

Why modifying it is security-relevant. The audit policy decides what defenders can see. An attacker who weakens it can perform later actions without leaving the events that a SIEM relies on. For that reason the change itself is a high-value signal.

What the commands do. Both commands specify only the failure setting of the subcategory. The first turns off logging of failed events for "Other Policy Change Events"; the second turns it on again. The test therefore represents a short, reversible audit policy change. It does not disable auditing as a whole (see Section 8).

Why Event ID 4719 is generated. Windows records Event ID 4719 ("System audit policy was changed") in the Security log whenever the system audit policy changes. The event identifies the account that made the change (Subject), the subcategory and the change applied (for example, Failure removed or added). According to the lab timeline in Section 4, two 4719 events were recorded, one for each command.

3.4 Step 3: Clear the Windows Security Event Log

The Security log was then cleared remotely, in the same session:

```
wevtutil.exe cl Security
```

Observed evidence: Figure 1 shows the command followed by a new prompt with no error message.

What the Security log contains. The Security log holds the audit records of the system: logons and logon failures, account and group changes, privilege use, audit policy changes and, when enabled, process creation and object access. It is a primary source in most Windows investigations.

Why attackers clear it. Clearing the log removes local evidence of the logons and actions that came before, and it can delay or complicate the investigation. Clearing a log requires administrative rights, which is why it usually follows a successful compromise of an administrator account.

Effect on forensic investigations. Once the local log is cleared, events cannot be recovered from the log itself unless it was saved or exported first. Investigators must then rely on copies held elsewhere, such as the SIEM, event forwarding, backups or disk artifacts. This is why shipping logs to Elastic quickly matters: events already ingested are not removed when the local log is cleared.

Why Event ID 1102 is generated. When the Security log is cleared, Windows writes Event ID 1102 ("The audit log was cleared") as the first entry of the cleared log. It records the account and Logon ID that cleared the log. In this event the account is stored in the UserData section, which is why the rule uses `winlog.user_data.SubjectUserName` for this step.

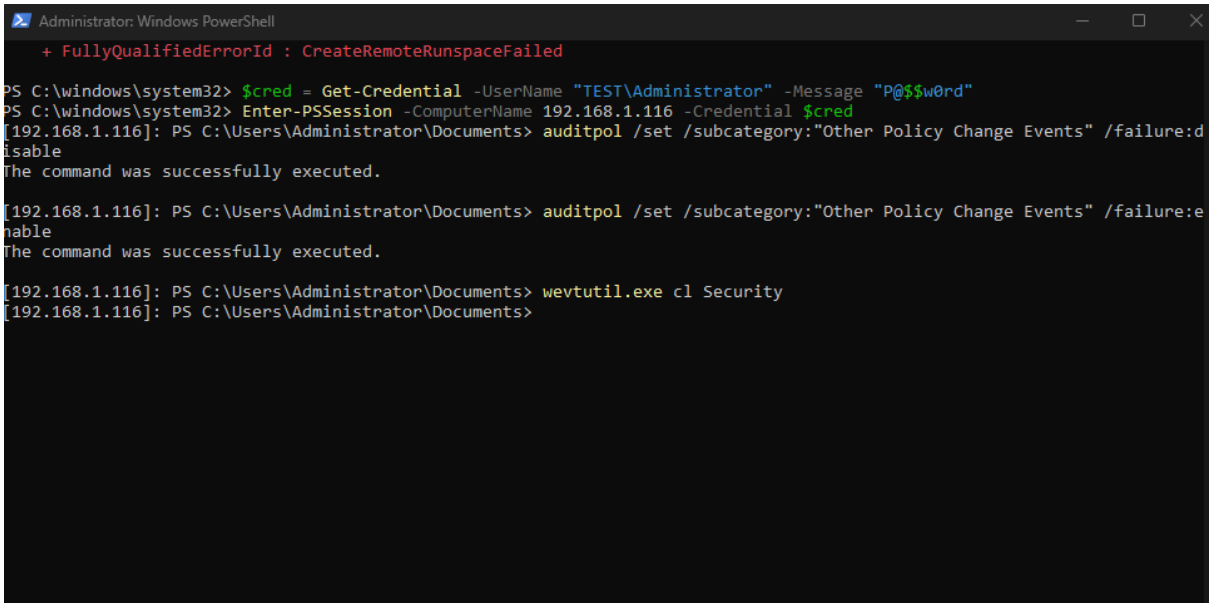


Figure 1 – Remote PowerShell session: credentials, WinRM session, audit policy commands and Security log clear

What Figure 1 proves: Figure 1 shows the whole command sequence of Steps 1 to 3 in a single session to 192.168.1.116 with TEST\Administrator.

What Figure 1 does not prove: Figure 1 does not show the Windows events generated by these commands, the source hostname, or any timestamps. Those are covered by Figures 3 to 6, which are pending. It does not show whether the earlier `CreateRemoteRunspaceFailed` error was a failed connection attempt or a different command.

4. Detection and Event Correlation

4.1 Detection workflow

Figure 2 shows how the simulated activity becomes an alert. The three Windows events are written to the Security log of the target, shipped by Elastic Agent to Elasticsearch, and evaluated by the EQL sequence rule on its schedule.

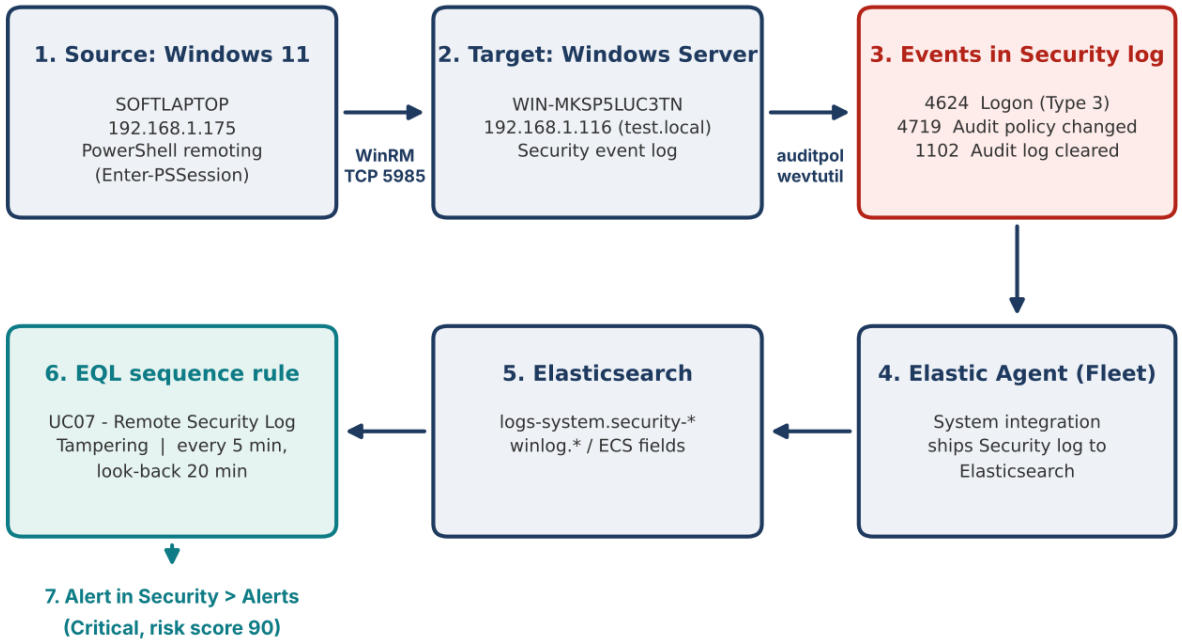


Figure 2 – Detection workflow for UC07

4.2 The three-event detection sequence

Order	Event ID	Description	Significance
1	4624	Successful Logon	Identifies the network logon associated with remote access
2	4719	System Audit Policy Changed	Detects modification of the audit policy
3	1102	Audit Log Cleared	Detects clearing of the Windows Security audit log

4.3 Evidence from the successful test

> **Evidence pending:** The values below were recorded by the lab team from Kibana Discover during the test. The corresponding screenshots (Figures 3 to 6) have not been provided yet, so these values are reported as supplied and are not backed by an image in this document.

Item	Value
Target host	win-mksp5luc3tn
User	Administrator
Source IP	192.168.1.175
Source workstation	SOFTLAPTOP

Item	Value
Logon type	3 (Network)
Shared Logon ID	0x6a7606

Timeline, November 2, 2026:

Event ID	Timestamp	Logon ID	Details	Elapsed since 4624
4624	00:53:13.198	0x6a7606	Successful network logon	0
4719	00:53:52.789	0x6a7606	Audit policy failure auditing disabled	+00:39.591
4719	00:53:57.334	0x6a7606	Audit policy failure auditing enabled	+00:44.136
1102	00:54:57.381	0x6a7606	Security audit log cleared	+01:44.183

Interpretation: All four events carry the same Logon ID, 0x6a7606. A Logon ID identifies one logon session on one host, so the shared value links the audit policy changes and the log clear to the network logon that opened the session. This is strong evidence that the three event types in the sequence belong to the same remote session.

The elapsed time column is calculated from the supplied timestamps. The full chain, from logon to log clear, took 1 minute 44 seconds, well inside the rule's 15-minute window.

Interpretation: A network logon (Logon Type 3) is consistent with WinRM but is not proof of it on its own, because other network services also produce Logon Type 3. In this lab, WinRM is confirmed by the procedure that was actually used: `Enter-PSSession` to the target over WinRM on TCP 5985 (Figure 1).

5. Elastic EQL Detection Rule

5.1 Rule summary

Setting	Value
Rule name	UC07 - Remote Security Log Tampering
Rule type	Event Correlation (EQL)
Index pattern	logs-system.security-*
Severity / Risk score	Critical / 90
Max alerts per run	100
Tags	Windows, Security, Remote-CLI, WinRM, Audit-Policy, Log-Tampering, Defense-Evasion, UC07
Description	Detects a sequence of remote network logon, Windows audit policy modification, and Security Event Log clearing on the same host and logon session. This activity may indicate an attempt to impair security monitoring and hide attacker activity.
False positive examples	Authorized system administrators performing scheduled audit policy changes or security log maintenance may generate similar events. Verify the source IP, account, logon session, and approved maintenance activity.
Created / Updated	Nov 2, 2026 @ 00:22:23.214 / Nov 2, 2026 @ 01:01:20.837 (by elastic)

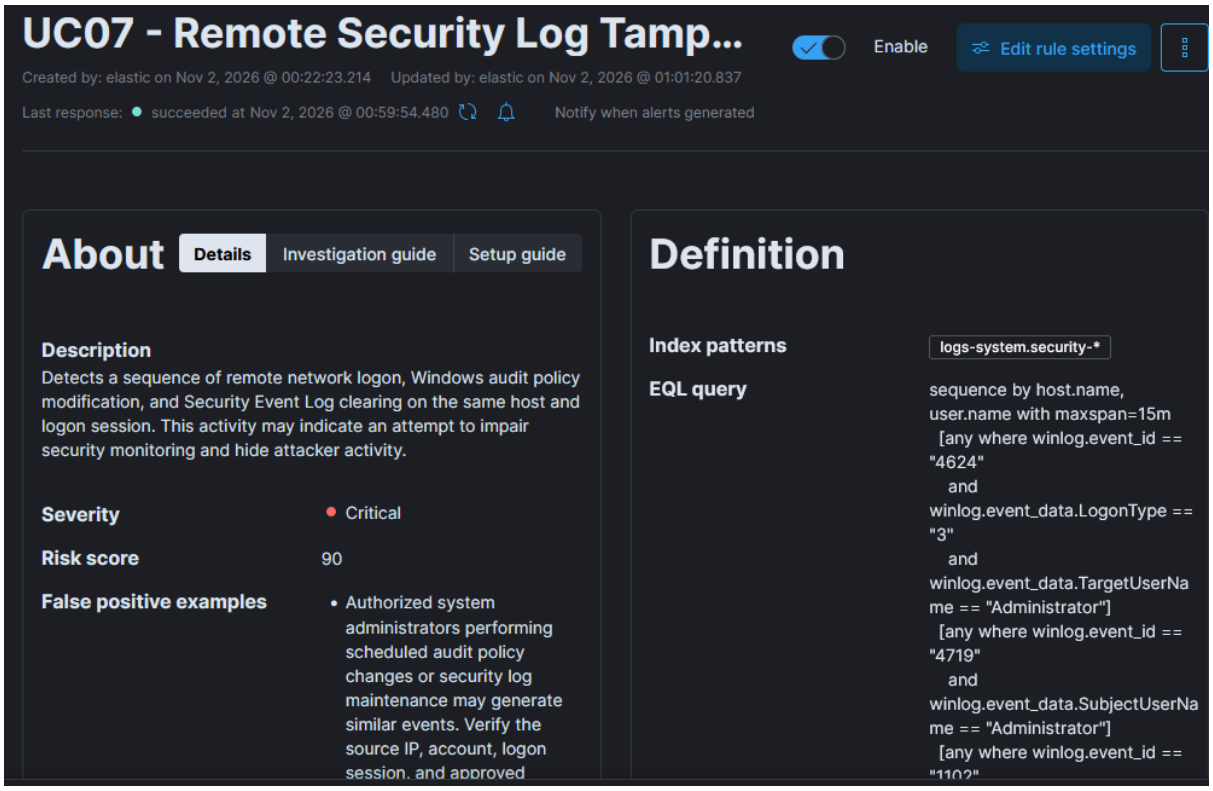


Figure 7 – UC07 - Remote Security Log Tampering: rule header, description and EQL definition

Observed evidence: Figure 7 shows the rule name, the Enabled toggle, the creation and update times, the last response (succeeded at Nov 2, 2026 @ 00:59:54.480), the description, severity Critical, risk score 90, the index pattern `logs-system.security-*` and the EQL query.

5.2 Final EQL query

```
sequence by host.name, user.name with maxspan=15m
  [any where winlog.event_id == "4624"
    and winlog.event_data.LogonType == "3"
    and winlog.event_data.TargetUserName == "Administrator"]
  [any where winlog.event_id == "4719"
    and winlog.event_data.SubjectUserName == "Administrator"]
  [any where winlog.event_id == "1102"
    and winlog.user_data.SubjectUserName == "Administrator"]
```

5.3 Query explained

Part of the query	What it does	Why it matters
`sequence`	Matches events in a defined order: each step must occur after the previous one.	The attack is a chain of actions. A sequence expresses the order that makes the chain suspicious.
`by host.name, user.name`	Join keys. Events are matched only when they have the same host.name and user.name values.	Prevents events from different machines or accounts from being combined into one match.
`with maxspan=15m`	The time from the first to the last event of the sequence must not exceed 15 minutes.	Keeps the match tied to one activity burst. The test sequence took 1 min 44 s.
Step 1: 4624, `LogonType == "3"`, `TargetUserName == "Administrator"`	A successful network logon by the Administrator account.	Represents the remote access that starts the sequence. The event ID is compared as a string.
Step 2: 4719, `SubjectUserName == "Administrator"`	An audit policy change performed by Administrator.	Represents weakening of the audit policy.
Step 3: 1102, `winlog.user_data.SubjectUserName == "Administrator"`	The Security log was cleared by Administrator. In event 1102 the account is stored in UserData.	Represents removal of evidence. This is the final step that completes the sequence.
`any where`	Matches the event without requiring a specific ECS event category.	Useful because all three steps are selected by Windows event ID.
Event order	4624, then 4719, then 1102.	A log clear without a preceding remote logon and policy change, or the reverse order, does not match. This reduces false positives.

5.4 Important technical note on Logon ID

Limitation: The final working rule correlates events using `host.name` and `user.name`. It does **not** enforce exact Logon ID matching inside the EQL query.

In the captured test evidence, the shared Logon ID (0x6a7606) independently confirms that the three events belonged to the same session in this simulation. This confirmation comes from the evidence, not from the rule.

Limitation: if the same account had two sessions on the same host within 15 minutes, events from different sessions could be matched together. The rule description says “same host and logon session”, which is stronger than what the query enforces.

5.5 Possible hardening (not implemented, not tested)

EQL allows a join key per step, so the Logon ID could be added to the correlation. The sketch below uses the field names expected for the three event types. It has not been run, and the field names must be checked against the ingested documents before use.

```
sequence by host.name with maxspan=15m
  [any where winlog.event_id == "4624" and winlog.event_data.LogonType == "3"]
      by winlog.event_data.TargetLogonId
  [any where winlog.event_id == "4719"] by winlog.event_data.SubjectLogonId
  [any where winlog.event_id == "1102"] by winlog.user_data.SubjectLogonId
```

6. Rule Scheduling Configuration

Setting	Value
Runs every	5 minutes
Additional look-back time	20 minutes
EQL maxspan	15 minutes

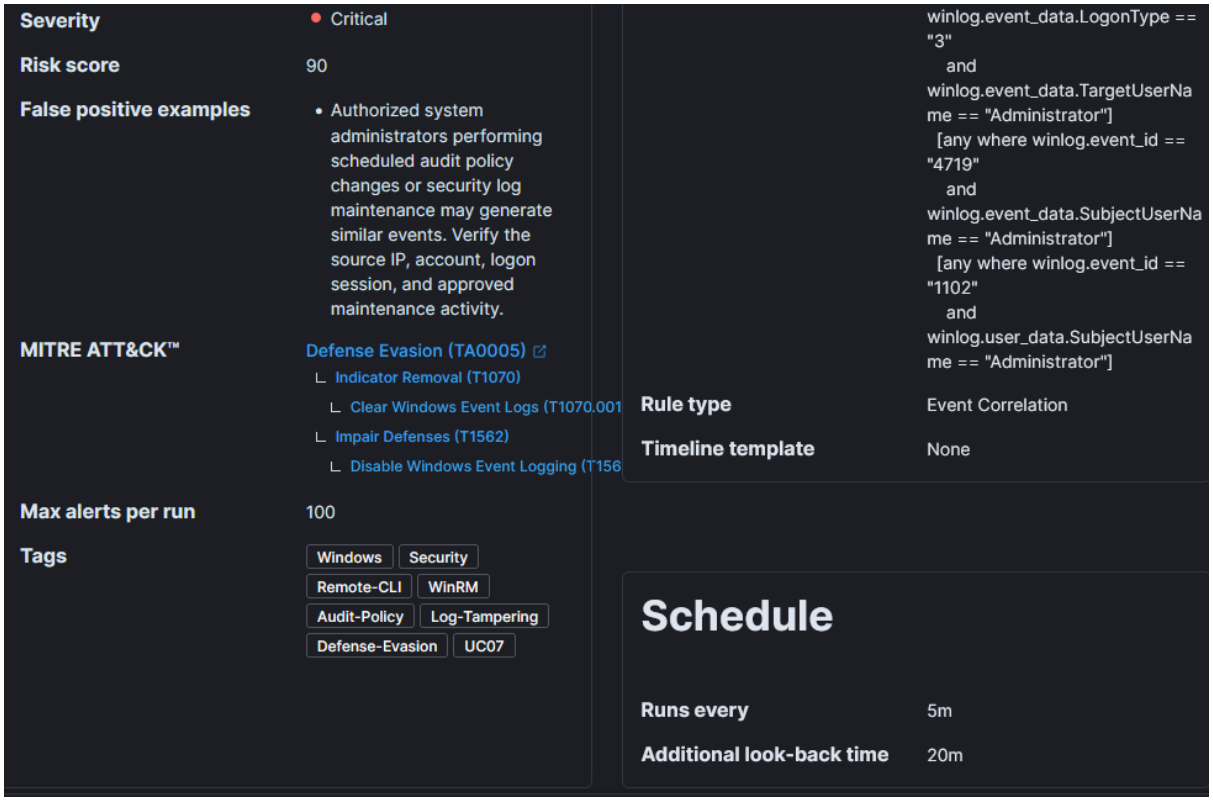


Figure 8 – UC07 rule details: severity, MITRE ATT&CK mapping, tags, rule type and schedule

Observed evidence: Figure 8 shows risk score 90, MITRE ATT&CK mapping (Defense Evasion TA0005; Indicator Removal T1070 with Clear Windows Event Logs T1070.001; Impair Defenses T1562 with Disable Windows Event Logging, whose ID is cut off at the edge of the capture), max alerts per run 100, the tags, rule type Event Correlation, timeline template None, and the schedule: Runs every 5m, Additional look-back time 20m.

6.1 Purpose of periodic execution and look-back

Elastic detection rules are scheduled searches. They do not evaluate each event as it arrives. Every 5 minutes the rule searches the Security data and, if the sequence matches, creates an alert.

The additional look-back time extends each search backwards so that events are not missed because of ingestion delay, a late run or a sequence that began before the previous run. Each run covers roughly the run interval plus the look-back time (about 25 minutes here). That is larger than the 15-minute maxspan, so a complete sequence still falls inside one search window.

> **Interpretation:** In the test, the 1102 event was recorded at 00:54:57.381 and the rule's last response in Figure 7 is 00:59:54.480. If the rule runs at a regular 5-minute interval, the previous run was at about 00:54:54, about 3 seconds before the log clear, so it could not have seen the complete sequence. The next run detected it, and the alert was created at 00:59:54.902 (Figure 9), about 4 minutes 57 seconds after the 1102 event. This delay is a result of the schedule. It is not a rule fault.

7. Detection Results and Validation

After the test sequence, the new rule **UC07 - Remote Security Log Tampering** was enabled and generated an alert.

7.1 Alert generated

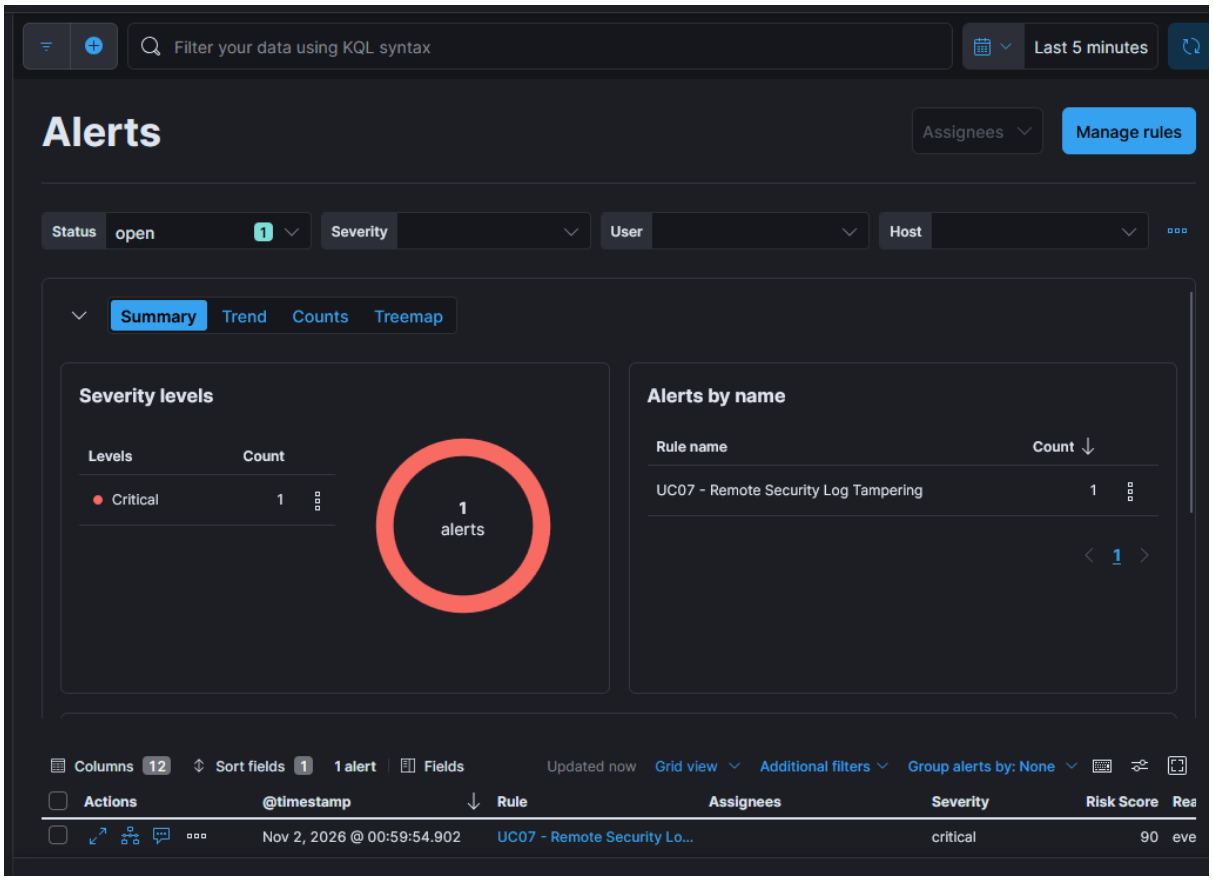


Figure 9 – Security > Alerts: one open Critical alert from UC07 - Remote Security Log Tampering

Field	Observed in Figures 7 and 9
Rule	UC07 - Remote Security Log Tampering (Alerts by name: count 1)
Alert timestamp	Nov 2, 2026 @ 00:59:54.902

Field	Observed in Figures 7 and 9
Severity / Risk score	Critical / 90
Status filter	open (1 alert)
Time range of the page	Last 5 minutes
Rule state	Enabled; last response succeeded at Nov 2, 2026 @ 00:59:54.480

7.2 What the alert confirms

> **Interpretation:** The alert shows that the rule definition in Figure 7 matched data in `logs-system.security-*` after the test, and that the detection pipeline works from the Security log of the target through Elastic Agent and Elasticsearch to the Alerts page. Because the rule is a sequence, an alert is expected only when a network logon, an audit policy change and a log clear are all matched in order for the same host and user within 15 minutes.

> **Note:** Figures 7 to 9 show the rule and the alert list. They do not show the source events behind the alert, or the host and user of the alert (the Reason column is cut off). The link between this alert and Event IDs 4624, 4719 and 1102 should be confirmed from the alert details and Figures 3 to 6.

7.3 Why correlation is better than detecting Event ID 1102 alone

- **Fewer false positives.** A log clear alone has legitimate causes such as maintenance or log rotation. The chain remote logon, audit policy change, log clear is far less likely to be routine.
- **More context.** The alert carries the story of the activity: how the actor arrived (remote network logon), what they did to reduce visibility, and what they removed.
- **Resilience to partial evidence.** If the log clear removes local events, the earlier events were already ingested in Elasticsearch and remain available for the investigation.
- **Better prioritization.** The Critical severity and risk score 90 are justified by the combination of behaviors, not by one event.

7.4 Value for a SOC analyst

The alert points the analyst to a host, an account and a short time window in which a remote session weakened auditing and removed log evidence. This is a defense-evasion pattern, and it often means that other malicious activity happened before it. The alert therefore starts a wider investigation (Section 9) and does not close it.

7.5 New rule versus the older rule

The older rule, **UC07 - Windows Security Log Tampering**, is a different rule. Its values below come from the earlier UC07 documentation of this project. The alert in Figure 9 is attributed to the new rule by the “Alerts by name” panel, and no alert from the older rule is shown or attributed to the new rule in this document.

Property	Older: UC07 - Windows Security Log Tampering	New: UC07 - Remote Security Log Tampering
Rule type	Query (KQL)	Event Correlation (EQL)
Logic	`winlog.event_id: ("4719" or "1102")`. Either event alone triggers an alert.	Ordered sequence of 4624 (Type 3), 4719 and 1102.

Property	Older: UC07 - Windows Security Log Tampering	New: UC07 - Remote Security Log Tampering
Correlation	None. No host, account or order check.	Same host.name and user.name within 15 minutes.
Risk score	73	90
Index pattern	logs-system.security-default	logs-system.security-*
Additional look-back	1 minute	20 minutes

8. MITRE ATT&CK Mapping

The mapping below separates what the simulation demonstrated from what it only partly supports. Technique IDs and names were checked against the Elastic prebuilt rule "Disable Windows Event and Security Logs Using Built-in Tools", which maps auditpol-based log disabling to T1562.002 and log clearing to T1070.001 under Defense Evasion (TA0005).

Tactic	Technique	Behavior in the simulation	Support level
Defense Evasion (TA0005)	T1070 Indicator Removal	Parent technique of the log clear.	Applies through T1070.001
Defense Evasion (TA0005)	T1070.001 Clear Windows Event Logs	`wevtutil.exe cl Security` (Figure 1). Windows records Event ID 1102.	Demonstrated. The command is shown; the 1102 event screenshot is pending (Figure 5).
Defense Evasion (TA0005)	T1562 Impair Defenses	The audit policy was modified with `auditpol`.	Applies through T1562.002
Defense Evasion (TA0005)	T1562.002 Disable Windows Event Logging	Failure auditing of one subcategory ("Other Policy Change Events") was disabled and then enabled again. Two 4719 events were recorded.	Partially supported. Only one subcategory's failure auditing was changed, briefly. Logging was not disabled as a whole.
Lateral Movement (TA0008)	T1021.006 Remote Services: Windows Remote Management	Remote PowerShell session to the target over WinRM, TCP 5985 (Figure 1).	Demonstrated by the procedure. Event 4624 Logon Type 3 alone is not proof of WinRM.

> **Interpretation:** The rule configuration in Figure 8 lists TA0005, T1070, T1070.001 and T1562 with a sub-technique of Impair Defenses whose ID is cut off. T1021.006 does not appear in that mapping. It is documented here as observed behavior. It should be added to the rule only if the detection is changed to identify WinRM specifically, because the current query matches a network logon, not the protocol.

9. SOC Analyst Investigation and Response

9.1 Investigation procedure

#	Step	How	Answer in this test
1	Identify the affected endpoint	Alert details: host.name, agent name, host.ip	WIN-MKSP5LUC3TN, 192.168.1.116
2	Identify the source IP and workstation	Event 4624: IpAddress and WorkstationName	192.168.1.175, SOFTLAPTOP (lab record)
3	Identify the account involved	TargetUserName (4624), SubjectUserName (4719, 1102)	Administrator (TEST\Administrator)
4	Review Event IDs 4624, 4719 and 1102	Discover, same host, around the alert time	Logon Type 3; two 4719 events; one 1102 event (lab record)
5	Correlate timestamps and Logon IDs	Sort by @timestamp and compare the Logon ID fields	Logon ID 0x6a7606 on all events; 1 min 44 s from logon to clear
6	Determine whether the activity was authorized	Ask the account owner and check change tickets and maintenance windows	Lab test. In production, treat as unauthorized until confirmed.
7	Preserve forensic evidence	Export the alert and the events from Elastic; snapshot or image the host; note that the local Security log was cleared	Events ingested before the clear remain in Elasticsearch
8	Escalate and contain	Follow the organization's incident response procedure (see 9.3)	Critical severity: escalate to the incident lead

9.2 Useful Discover queries

Field names are the ones expected from the System integration. Confirm them against the ingested documents, especially the Logon ID fields.

```
// All three events on the affected host
host.name: "win-mksp5luc3tn" and winlog.event_id: ("4624" or "4719" or "1102")

// All events of the same logon session
winlog.event_data.TargetLogonId: "0x6a7606" or winlog.event_data.SubjectLogonId:
"0x6a7606"
```

9.3 Containment and response

- Confirm with the owner of the Administrator account whether the remote logon, the audit policy change and the log clear were authorized.
- If unauthorized, isolate the host and the source machine, reset the account credentials and end active sessions.
- Restore the audit policy to the approved baseline.
- Review other activity from the same account, source IP and host before and after the event, because log tampering usually follows other actions.
- Check other hosts for logons from the same source IP.
- Record the findings and the timeline in the case, and escalate according to the severity.

9.4 Recommended defensive measures

- **Restrict remote administration:** allow WinRM only from approved management hosts, prefer HTTPS (5986), and use JEA or a privileged access workstation.
- **Protect administrator accounts:** avoid routine use of the built-in Administrator account, use unique admin accounts with multi-factor authentication, and restrict network logon rights.
- **Protect the audit policy:** manage it by Group Policy and alert on any 4719 event outside an approved change.
- **Protect the logs:** forward logs to the SIEM in near real time, restrict who can clear logs, and alert on 1102 events.
- **Improve the rule:** correlate by Logon ID as in Section 5.5, and add a companion rule for log clearing or audit policy changes by any account.
- **Run the test only in an isolated lab,** with a documented procedure.

UC08

KERBEROASTING DETECTION

Elastic Security SIEM | Windows Active Directory Lab

Technical Implementation & Validation Report

Document information	Value
Project	Project 4: SIEM Detection Engineering, Alert Triage & SOC Operations
Program	Digital Egypt Pioneers Initiative
Use case	UC08 – Kerberoasting Detection (RC4)
Severity / Risk score	High / 73
SIEM platform	Elastic Security / Kibana 8.19.x
Detection type	KQL custom query rule
Test date	November 5, 2026
Prepared by	Ahmed Mohamed Ahmed Abdo
Document status	Rule created and enabled; alert validation pending

Purpose: document the setup, controlled Kerberos ticket requests, detection logic, rule configuration, and current test evidence. This report distinguishes completed actions from items that still require validation.

1. Executive Summary

This use case was developed to monitor Windows Kerberos Service Ticket Request events (Event ID 4769) and identify requests for the lab SQL service that use RC4-HMAC encryption. The intended policy is account-based: **TEST\sql-ahmed** is the authorized account and should not generate an alert, while a different domain account requesting the same SQL service should generate an alert.

The service account and SPN were configured, a service ticket was successfully requested from the Windows 10 client, and a Custom query detection rule was created in Elastic Security. A later test using **TEST\ahmed.mohamed** also successfully obtained the SQL service ticket with RC4-HMAC. However, the Elastic Alerts page showed no matching alerts at the time of documentation. Therefore, end-to-end alert generation is not yet confirmed.

Important scope note: requesting a Kerberos service ticket is a controlled simulation step and a useful detection signal. It does not, by itself, prove that a complete Kerberoasting attack occurred. Ticket extraction and offline password-cracking were not performed.

2. Objective and Detection Logic

Objective

Detect successful Event ID 4769 requests for the SQL service when the requesting account is not the approved account TEST\sql-ahmed, with RC4-HMAC ticket encryption (0x17).

Expected behavior

Requesting account	Requested service	Expected result
TEST\sql-ahmed	SQL service SPN	No alert (authorized account)
TEST\ahmed.mohamed	SQL service SPN	Alert (unauthorized account in this lab policy)
Any other non-allowlisted account	SQL service SPN	Alert, subject to event/field matching

The rule must match the actual event fields emitted by the Domain Controller. The requester identity and service-account identity are different concepts and must not be confused when configuring the KQL query.

3. Lab Environment

Component	Role / details
Windows Server 2019	Active Directory Domain Controller; issues Kerberos service tickets and records Security Event 4769.
Windows 10	Domain-joined client used to request the SQL service ticket.
Domain	TEST.LOCAL
Elastic SIEM	Elastic Security / Kibana, accessed at http://192.168.1.84:5601
Service SPN	MSSQLSvc/sqlserver.test.local:1433
Authorized service account	TEST\sql-ahmed
Test requester account	TEST\ahmed.mohamed

4. Implementation Procedure

4.1 Create the domain service account

A dedicated domain account named **TEST\sql-ahmed** was created on the Windows Server 2019 Domain Controller to represent the SQL service identity in the lab.

4.2 Register the Service Principal Name (SPN)

The following SQL service SPN was registered for the service account:

```
MSSQLSvc/sqlserver.test.local:1433
```

The SPN was then checked to confirm that it was associated with the intended service account. No full SQL Server installation or running SQL Server service was required for this controlled Kerberos ticket-request simulation.

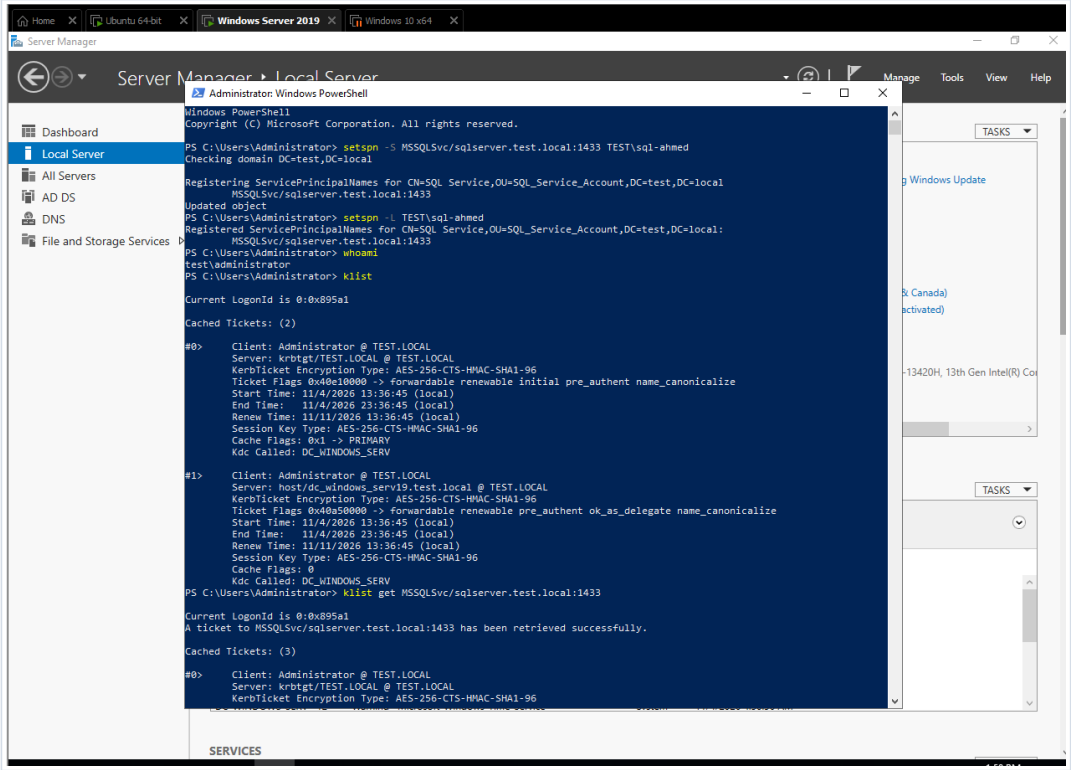


Figure 1 – SPN MSSQLSvc/sqlserver.test.local:1433 registered with setspn -S and checked with setspn -L for TEST\sql-ahmed (PowerShell on the Domain Controller, whoami: test\administrator).

4.3 Request the Kerberos service ticket

On the Windows 10 client, while signed in as TEST\sql-ahmed, the following command was executed:

```
klist get MSSQLSvc/sqlserver.test.local:1433
```

The command returned a successful ticket retrieval message. The cached ticket showed:

Field	Observed value
Client	sql-ahmed @ TEST.LOCAL
Server	MSSQLSvc/sqlserver.test.local:1433 @ TEST.LOCAL
Ticket encryption type	RSADSI RC4-HMAC(NT)
KDC	DC_windows_serv19.test.local

This confirms that the KDC issued a service ticket for the SQL SPN using RC4-HMAC. The result is evidence of ticket issuance, not evidence of ticket extraction or offline cracking.

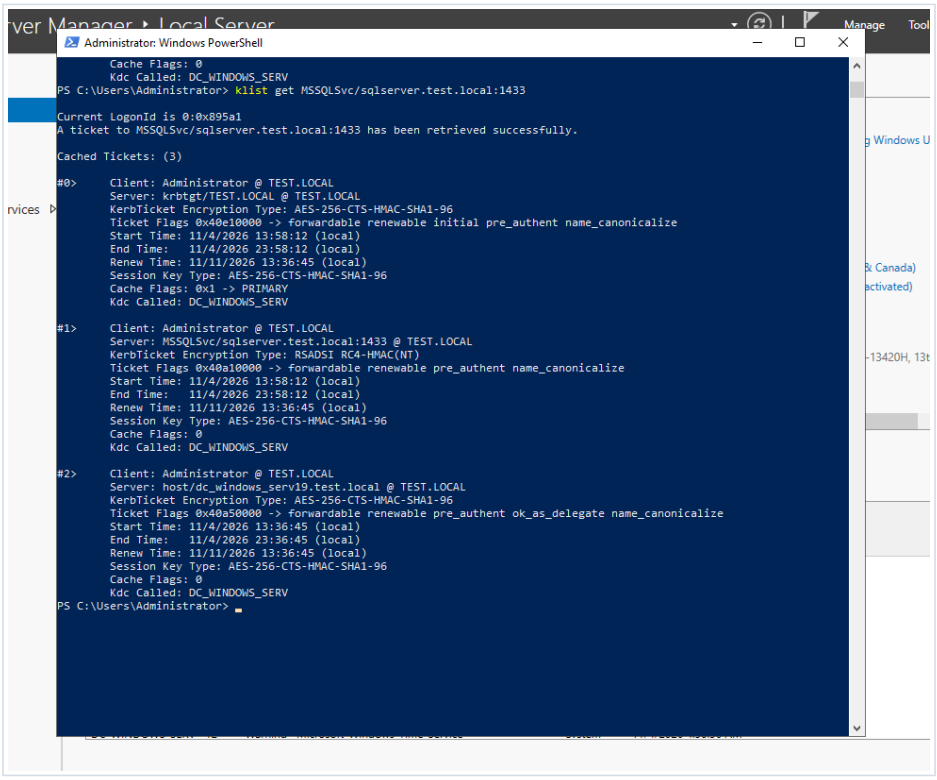


Figure 2 – klist get MSSQLSvc/sqlserver.test.local:1433 run on the Domain Controller as TEST\Administrator: the cached ticket for the SQL SPN shows RSADSI RC4-HMAC(NT).

4.4 Confirm Windows Security event collection

Event ID 4769 was observed in Elastic from the Domain Controller, demonstrating that Kerberos Service Ticket Request events were being ingested. An earlier 4769 event showed host **dc_windows_serv19**, action **kerberos-service-ticket-requested**, outcome **success**, and encryption value **0x12**. That event was not confirmed as the specific SQL SPN test event, so it was not treated as proof of the RC4 SQL request.

A precise query for the SQL SPN did not initially return a result. The next validation step is to inspect the complete raw JSON for the event generated at the time of each test and confirm the exact requester, service, encryption, status, and source-address fields.

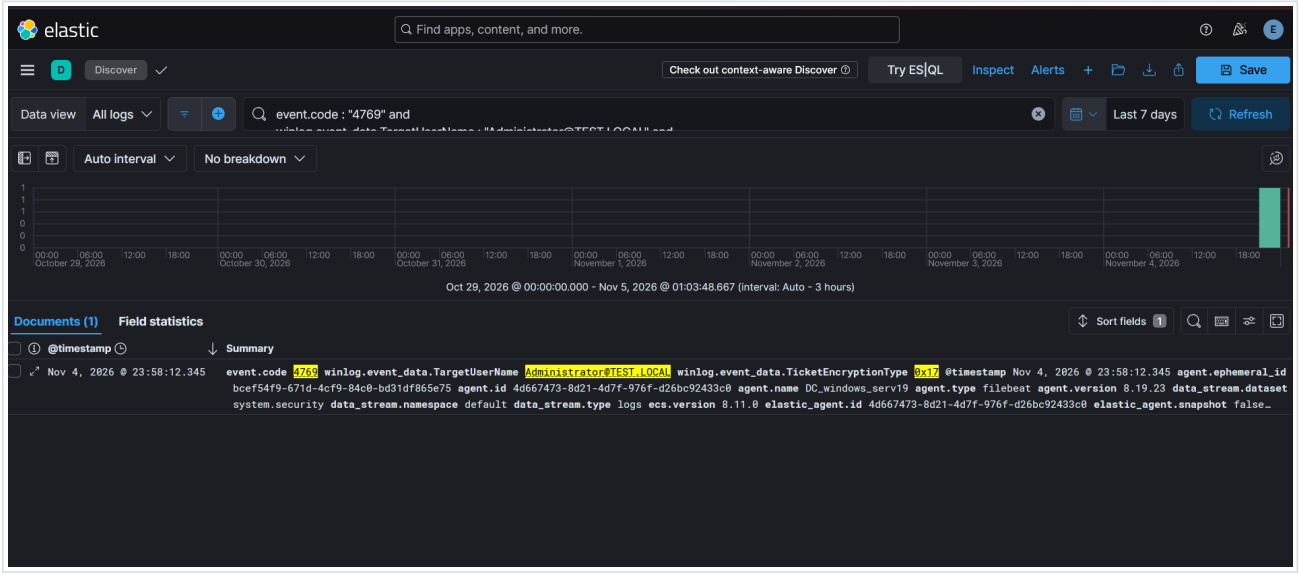


Figure 3 – Discover: Event ID 4769 with TargetUserName Administrator@TEST.LOCAL and TicketEncryptionType 0x17 (Nov 4, 2026 @ 23:58:12.345, as displayed in Kibana).

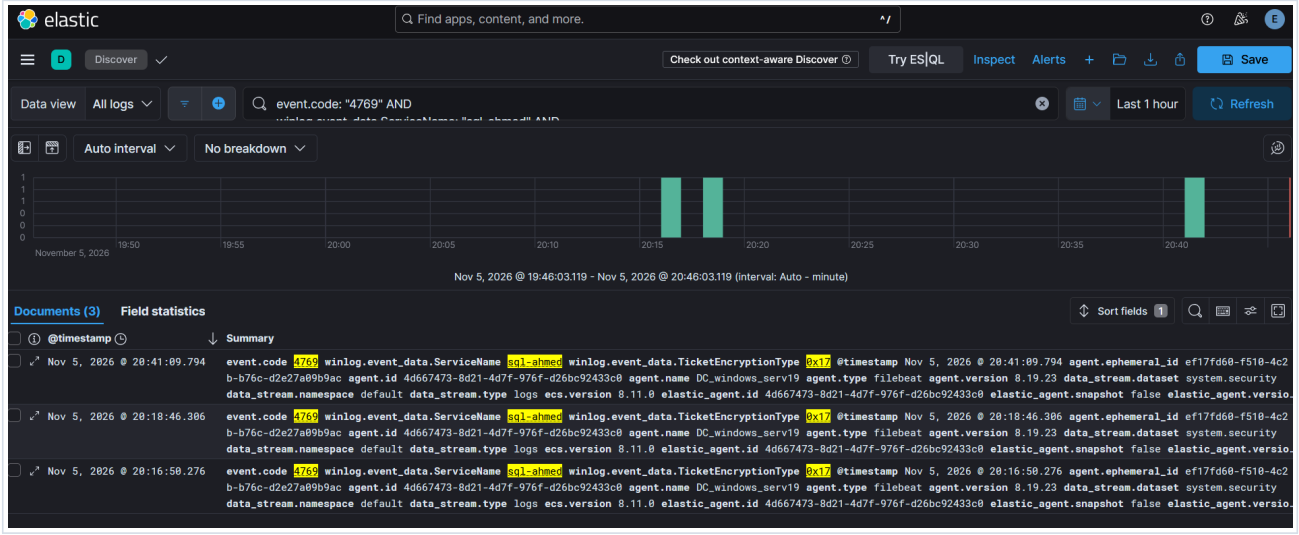


Figure 4 – Discover: Event ID 4769 documents with ServiceName sql-ahmed and TicketEncryptionType 0x17 (3 documents, last hour, Nov 5, 2026 20:16–20:41). The query text is truncated in the capture.

5. Elastic Detection Rule Configuration

Setting	Configured value
Rule name	UC08 - Kerberoasting Detection (RC4)
Created by	elastic on Nov 5, 2026 @ 20:59:35.566
Updated by	elastic on Nov 5, 2026 @ 21:30:12.533
Last response	succeeded at Nov 5, 2026 @ 21:54:40.655
Description	Detects successful Kerberos service ticket requests using RC4 encryption, which may indicate Kerberoasting activity
Severity	High
Risk score	73
False positive examples	Legitimate legacy applications or services using RC4 encryption for Kerberos authentication may trigger this detection. Validate the requesting account, service, source host, and business justification.
MITRE ATT&CK	Credential Access (TA0006) Steal or Forge Kerberos Tickets (T1558) Kerberoasting (T1558.003)
Max alerts per run	100
Tags	UC08, Kerberoasting, Kerberos, Active-Directory, RC4, Credential-Access
Index patterns	apm-*-transaction*, auditbeat-*, endgame-*, filebeat-*, logs-*, packetbeat-*, traces-apm*, winlogbeat-*, -*elastic-cloud-logs-*
Custom query	`event.code: "4769" AND winlog.event_data.TicketEncryptionType: "0x17" AND winlog.event_data.Status: "0x0" AND winlog.event_data.ServiceName: "sql-ahmed" AND NOT winlog.event_data.TargetUserName: "sql-ahmed@TEST.LOCAL"`
Custom query language	KQL
Rule type	Query
Timeline template	None
Schedule: runs every	5m (5 minutes)
Additional look-back time	1m (1 minute)

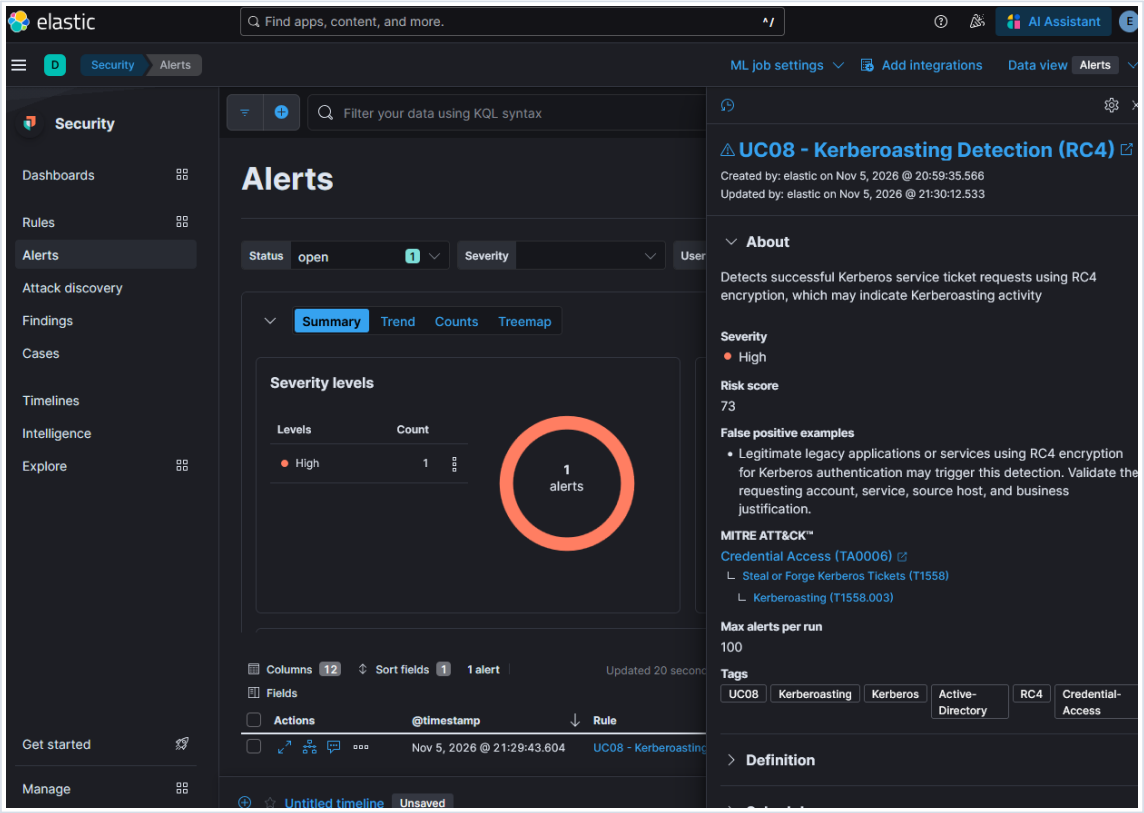


Figure 5 – Rule details in Elastic Security: description, severity High, risk score 73, MITRE ATT&CK mapping (TA0006, T1558, T1558.003) and tags (created Nov 5, 2026 @ 20:59:35.566; updated Nov 5, 2026 @ 21:30:12.533).

6. Current KQL Definition

The rule details page showed the following query at the time of this report:

```
event.code: "4769" AND
winlog.event_data.TicketEncryptionType: "0x17" AND
winlog.event_data.Status: "0x0" AND
winlog.event_data.ServiceName: "sql-ahmed" AND
NOT winlog.event_data.TargetUserName: "sql-ahmed@TEST.LOCAL"
```

Validation caution: this query is the currently configured query, not yet a verified final query. The exact values and mappings for ServiceName, TargetUserName, TicketEncryptionType, and Status must be confirmed from the raw 4769 document generated by the test. In particular, confirm whether ServiceName contains the service account name or the SPN, and whether TargetUserName contains the requester in the format used by the exclusion.

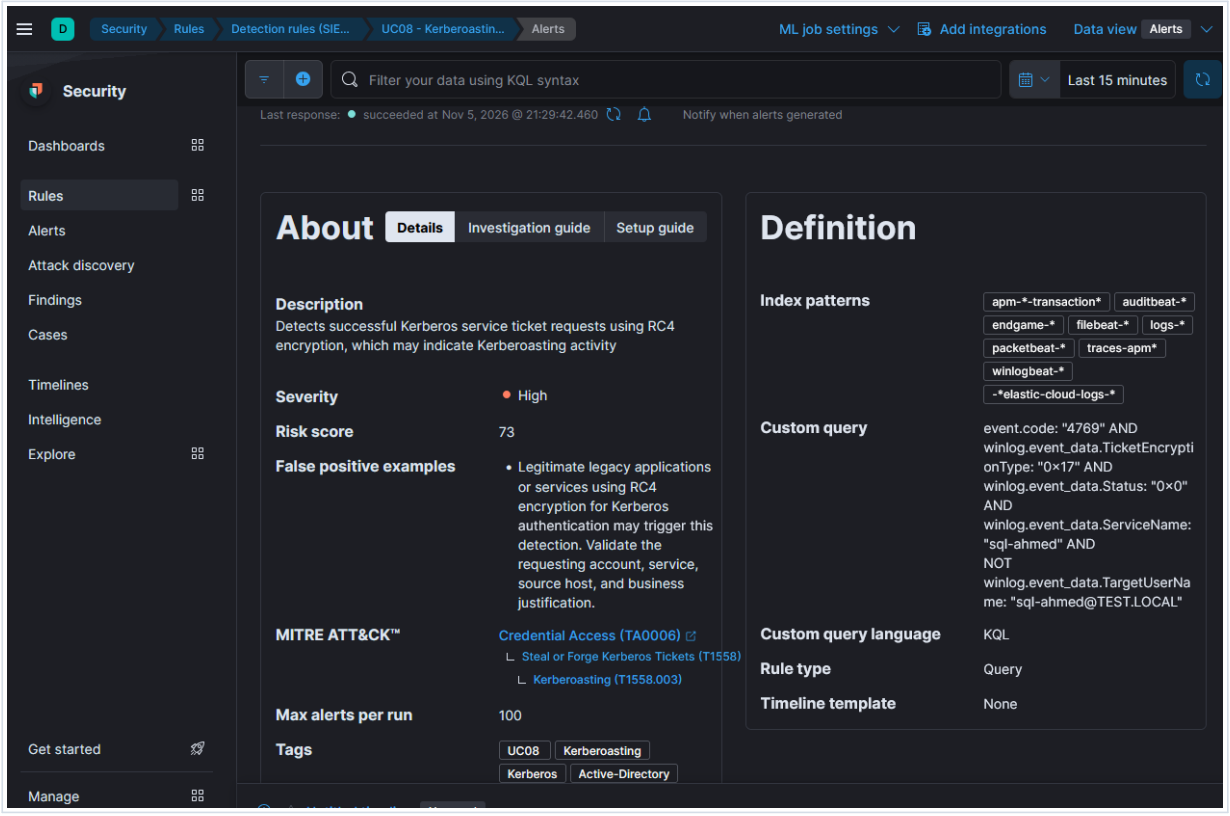


Figure 6 – Rule definition page: index patterns, custom query (KQL), rule type Query; last response succeeded at Nov 5, 2026 @ 21:29:42.460.

7. Test Execution and Results

7.1 Authorized-account test

The authorized account TEST\sql-ahmed previously executed the SQL SPN request successfully. The returned service ticket used RC4-HMAC. The intended result for this account is no alert.

7.2 Non-allowlisted-account test

A second test was performed on Windows 10 while signed in as **TEST\ahmed.mohamed**. The ticket cache was cleared and the SQL service ticket was requested again:

```
klist purge
klist get MSSQLSvc/sqlserver.test.local:1433
whoami
```

Observed results:

Evidence	Observed result
Ticket purge	Ticket(s) purged!
Ticket request	A ticket to MSSQLSvc/sqlserver.test.local:1433 has been retrieved successfully.
Requester identity	test\ahmed.mohamed
Ticket encryption	RSADSI RC4-HMAC(NT)
KDC	DC_windows_serv19.test.local

This is the intended positive test scenario under the lab's allowlist policy: a non-allowlisted domain account successfully requested the SQL service ticket using RC4-HMAC.

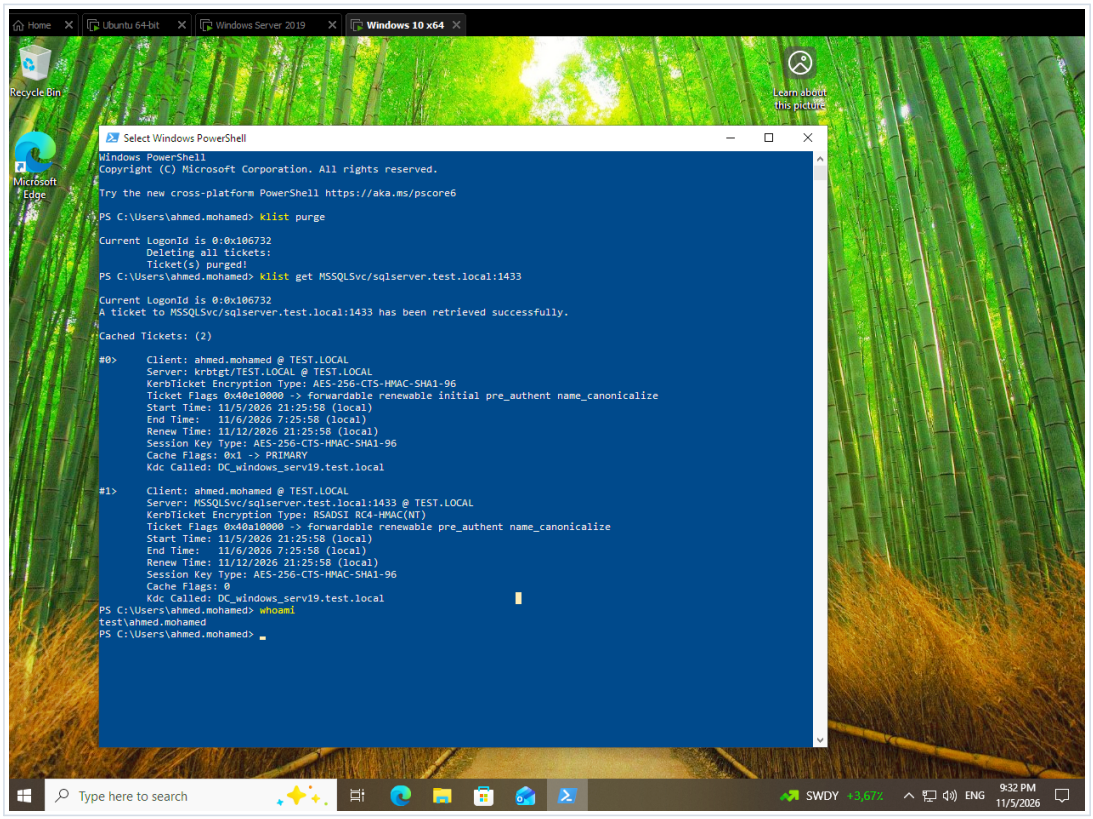


Figure 7 – Windows 10 client as test\ahmed.mohamed: klist purge, klist get MSSQLSvc/sqlserver.test.local:1433 (ticket retrieved, RSADSI RC4-HMAC(NT)) and whoami.

7.3 Alert verification

The Elastic rule details page showed a successful last execution response at 21:54:40 on 05 November 2026. However, the rule's Alerts view and the Security → Alerts page showed no matching results in the selected time range. Therefore, the detection rule has been created and its execution succeeded, but alert generation for the non-allowlisted account has not yet been demonstrated.

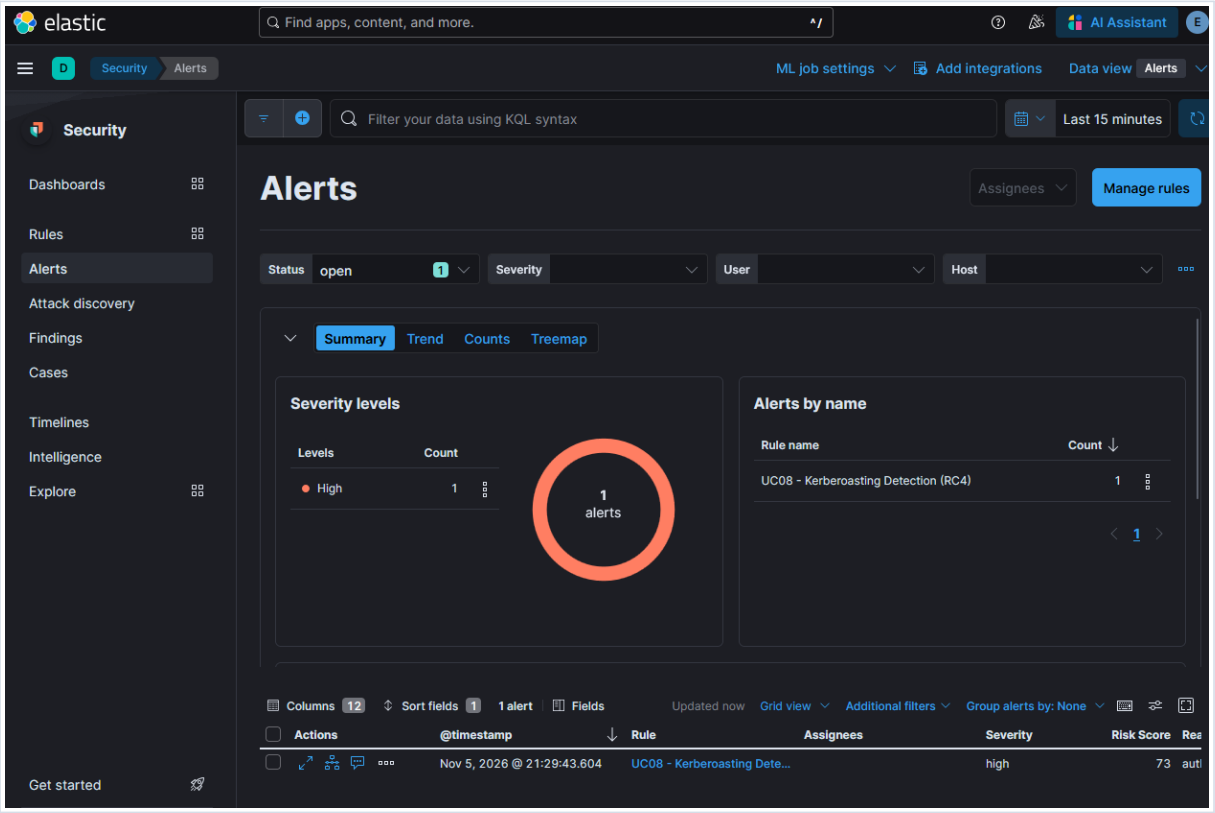


Figure 8 – Security → Alerts: one open High alert from UC08 - Kerberoasting Detection (RC4), risk score 73, timestamp Nov 5, 2026 @ 21:29:43.604.

Validation item	Status
Service account and SPN configured	Completed
SQL SPN ticket request successful	Completed
RC4-HMAC observed in ticket cache	Completed
Event ID 4769 ingestion observed	Completed (general event ingestion)
Exact SQL test event correlated in Elastic	Pending
Detection rule created and enabled	Completed
Rule execution response	Succeeded
Authorized-account no-alert test	Pending formal verification in Alerts
Non-allowlisted-account alert	Pending

8. Remaining Work

The following actions are required to complete end-to-end validation:

1. In Discover, search for **event.code: "4769"** around the exact test time and inspect the raw JSON for the request made by TEST\ahmed.mohamed.

2. Confirm the actual field names and values for the requester account, SQL service identity/SPN, ticket encryption type, request status, and client IP address.

3. Update the KQL to match the verified event fields. The rule should match the SQL service and RC4 success condition, while excluding only the approved requester TEST\sql-ahmed.

4. Use the rule preview or an equivalent Discover query to verify that the non-allowlisted event matches and the authorized event does not match.

5. Confirm the rule is enabled, wait for the next scheduled execution, and verify that an alert appears under Security → Alerts.

6. Record both test outcomes: authorized account = no alert; non-allowlisted account = alert.

9. Security and Detection Considerations

• RC4 usage is a useful detection signal but is not sufficient by itself to prove malicious Kerberoasting. Some legacy services may legitimately use RC4.

• The allowlist should be based on the requesting account field, not the service account field. The SQL service identity and the user requesting its ticket are separate fields in Event 4769.

• A production-quality detection can be strengthened with behavioral context, such as unusually high ticket-request volume, requests for multiple service accounts, unusual source hosts, and correlation with other suspicious activity.

• The lab's account policy is a controlled test assumption: TEST\sql-ahmed is authorized; TEST\ahmed.mohamed is treated as non-allowlisted for this exercise.
